

单位代码： 10293 密 级：

南京邮电大学

硕士学位论文



论文题目： 云原生环境下容器部署加速策略研究与实现

学 号	<u>1021072316</u>
姓 名	<u>顾加伟</u>
导 师	<u>张登银</u>
学 科 专 业	<u>信息网络</u>
研 究 方 向	<u>云计算</u>
申请学位类别	<u>工学硕士</u>
论文提交日期	<u>2024.06</u>

Research and Implementation of Container Deployment Acceleration Strategy in Cloud-Native Environments

Thesis Submitted to Nanjing University of Posts and
Telecommunications for the Degree of
Master of Science in Engineering



By

Gu Jiawei

Supervisor: Prof. Dengyin Zhang

June 2024

摘要

近年来，随着计算机技术的不断发展，企业对于大规模应用服务的部署及管理需求日益增加。在此背景下，容器技术凭借其轻量、可移植、便于扩展等优势，在业界得到了广泛使用。在实际应用中，容器部署速度直接关系到服务上线和更新迭代的效率，进而影响着服务质量，而镜像下载时延是决定容器部署速度的关键因素。因此，本文面向容器集群场景，围绕减少镜像下载量和提升镜像传输效率两个方面展开研究，旨在降低镜像下载时延，加速容器部署。本文主要贡献包含以下三个方面：

1) 现有的本地存储优先策略通过将任务调度到拥有完整所需镜像的节点以复用缓存，但此方法在节点仅拥有部分镜像层的场景下，忽略了不同镜像间可复用的镜像层，导致了不必要的下载。针对此问题，本文提出了一种基于节点镜像层缓存的容器调度方案。该方案将节点镜像层缓存信息预先存储在集群中，在调度任务来临时调用上述信息并与任务所需镜像层进行对比，进而将任务调度到缓存命中率最高的节点上。实验结果表明，该方案能够有效减少镜像下载量，从而加速容器部署。

2) 现有方法通过 P2P 传输技术处理并发的镜像拉取请求以提升镜像分发速度，但其传输初始阶段效率较低，难以适用于部分节点需要更快响应的任务场景。针对此问题，本文提出了一种基于 IPFS 的分批式镜像分发方案。该方案首先将集群中拥有所需镜像的节点加入到 IPFS 网络中，随后依据当前网络中的源节点数量，分批将需求节点加入 IPFS 网络进行镜像传输，直至所有需求节点都获取到所需镜像。实验结果表明，该方案能够有效提升节点的平均响应速度，并且提高了分发过程的鲁棒性。

3) 本文设计并实现了一种基于 Kubernetes 的容器部署多阶段加速系统。该系统在集群网络负载较低时，对热门镜像进行预拉取；在调度阶段，根据任务需求，综合考虑节点缓存、资源利用率和负载均衡因素，选取最优节点执行任务；在镜像分发阶段，根据不同集群环境和任务类型，自适应调整镜像分发方式，并在初始节点选择以及镜像 P2P 传输单位上进行优化。在每次部署任务完成后，系统通过日志的形式向用户反馈容器部署过程中的各项系统指标，并提供了统一接口供用户快捷地对系统参数进行调优。实验结果表明，该系统能够在灵活满足多种任务需求的同时有效提升容器的部署速度。

关键词： 镜像分发，镜像加速，容器调度，P2P 传输，IPFS

Abstract

In recent years, with the continuous development of computer technology, enterprises have increasing demand for large-scale service deployment and management. In this context, container technology has been widely used in the industry by virtue of its lightweight, portable, easy to expand and other advantages. In practical applications, container deployment speed is directly related to the efficiency of service on-line and update iteration, which in turn affects the service quality, and the image download delay is a key factor in determining the container deployment speed. Therefore, this thesis is oriented to container cluster scenarios, and focuses on reducing the amount of image downloads and improving the efficiency of image transmission, aiming at reducing the delay of image downloads and accelerating the deployment of containers. The main contributions of this thesis include the following three aspects:

1) Existing local storage prioritization strategy multiplexes the cache by scheduling tasks to nodes that have the complete required images, but this method ignores the image layers that can be multiplexed between different images in scenarios where the nodes have only some of the image layers, leading to unnecessary downloads. To address this problem, this thesis proposes a container scheduling scheme based on node image layer cache. The scheme stores the node image layer cache information in the cluster in advance, calls the above information when scheduling a task and compares it with the required image layer of the task, and then schedules the task to the node with the highest cache hit rate. Experimental results show that this scheme can effectively reduce the amount of image downloads, thus accelerating container deployment.

2) Existing methods handle concurrent image pull requests through P2P transmission technology to improve the speed of image distribution, but the initial phase of its transmission is less efficient, making it difficult to apply to task scenarios in which some nodes need to respond faster. To address this problem, this thesis proposes a batch-type image distribution scheme based on IPFS. The scheme first adds the nodes in the cluster that have the required images to the IPFS network, and then, based on the number of source nodes in the current network, adds the demand nodes to the IPFS network in batches for image transmission until all the demand nodes have obtained the required images. Experimental results show that this scheme can effectively improve the average response speed of the nodes, and improve the robustness of the distribution process.

3) This thesis designs and implements a multi-stage acceleration system for container deployment based on Kubernetes. The system pre-pulls popular images when the cluster network load is low; in the scheduling phase, the optimal node is selected to execute the task according to the task demand, taking into account the node cache, resource utilization, and load balancing factors; in the image distribution phase, the image distribution method is adaptively adjusted according to different cluster environments and task types, and optimized in the initial node selection as well as in the image P2P transmission unit. After the completion of each deployment task, the system provides feedback to the user in the form of logs on the system indicators during container deployment, and provides a unified interface for the user to quickly tune the system parameters. Experimental results show that the system can flexibly meet the needs of a variety of tasks while effectively improving the deployment speed of containers.

Key words: image distribution, image acceleration, container scheduling, P2P transmission, IPFS.

目 录

第一章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状分析.....	2
1.3 本文研究内容及章节安排	5
第二章 相关技术综述	8
2.1 云原生技术.....	8
2.2 容器技术.....	10
2.2.1 容器镜像.....	11
2.2.2 镜像分发.....	13
2.3 主流容器编排引擎.....	14
2.3.1 Apache Mesos	14
2.3.2 Docker Swarm.....	15
2.3.3 Kubernetes.....	16
2.4 P2P 传输技术	18
2.4.1 比特流协议 BitTorrent	18
2.4.2 星际文件系统 IPFS.....	20
2.5 本章小结.....	21
第三章 基于节点镜像层缓存的容器调度方案	22
3.1 背景分析.....	22
3.2 基于节点镜像层缓存的调度算法设计	23
3.3 基于节点镜像层缓存的调度框架设计	25
3.3.1 信息存储组件.....	27
3.3.2 缓存更新组件.....	28
3.3.3 评分调度组件.....	31
3.4 实验与分析.....	33
3.4.1 实验环境.....	33
3.4.2 镜像层缓存未命中场景.....	34
3.4.3 镜像层缓存部分命中场景.....	34
3.4.4 镜像层缓存完全命中场景.....	36
3.4.5 时延对比与总结.....	36
3.5 本章小结.....	38
第四章 基于 IPFS 的分批式镜像分发方案	39
4.1 背景分析.....	39
4.1.1 基于 BitTorrent 的镜像分发.....	40

4.1.2 基于 IPFS 的镜像分发.....	40
4.2 基于 IPFS 的分批式镜像分发策略设计	42
4.3 基于 IPFS 的分批式镜像分发框架设计	44
4.4 实验与分析.....	46
4.4.1 实验环境.....	46
4.4.2 结果对比与总结.....	47
4.5 本章小结.....	49
第五章 基于 Kubernetes 的容器部署多阶段加速系统.....	50
5.1 需求分析.....	50
5.1.1 系统功能性需求.....	50
5.1.2 系统非功能性需求.....	51
5.2 系统设计与实现.....	52
5.2.1 镜像预拉取模块.....	52
5.2.2 容器调度模块.....	54
5.2.3 镜像分发模块.....	56
5.2.4 反馈与调节模块.....	58
5.3 实验与分析.....	59
5.3.1 实验环境.....	59
5.3.2 实验结果对比.....	60
5.4 本章小结.....	63
第六章 总结与展望	64
6.1 论文研究工作总结.....	64
6.2 进一步研究工作展望.....	65
参考文献.....	66

第一章 绪论

1.1 研究背景与意义

随着互联网技术的进步和数字化转型的深入^[1],企业所提供的应用服务数量迅速增加,随之而来的是业界对资源优化、安全性以及隔离性等方面的需求显著提升^[2-4]。在过去的几十年中,虚拟化^[5-6]一直是企业级应用^[7]和数字服务中心^[8]的关键技术。虚拟化技术通过在一台物理主机上划分计算资源、模拟多个独立的虚拟环境,使得应用程序能够在隔离的环境中运行。然而,传统虚拟机存在资源消耗大、管理复杂和启动时间长等问题^[9-11],限制了其在云环境中的广泛应用。同时在近些年,云计算、大数据^[12]、人工智能^[13]等领域更侧重于计算资源的高效性、灵活性、可扩展性。在此背景下,以 Docker 为代表的容器化技术应运而生,成为了应对复杂应用部署和管理挑战的关键工具^[14]。

容器化技术通过将应用程序及其所需依赖项打包到一个独立的单元中,实现了应用的跨平台与一致性部署,减少了开发、测试和生产环境之间的差异,降低了因环境问题引起的错误^[15]。其通过共享操作系统内核,显著提高了资源利用率和部署效率。相较于传统虚拟化,容器具有更快的启动时间,使得应用可以在秒级别内启动,极大提高了服务的响应速度^[16]。此外,容器的轻量化设计降低了资源消耗,使得在同一物理机上可以运行更多的应用实例。

然而,随着容器数量的增加和应用复杂度的上升,单纯的容器技术已经无法满足应用管理和协调的复杂需求^[17]。为解决这一问题,业界纷纷采用容器编排引擎作为管理和协调集群中容器实例的工具。容器编排引擎提供了一个高效、灵活且自动化的平台以管理容器化应用的生命周期和资源,其通过对复杂的底层基础设施进行抽象化,使开发者和系统管理员能够以声明式配置和自动化的方式部署、扩展、更新和维护运行在容器中的应用,保障了应用的高可用性。容器编排引擎不仅大幅简化了应用的部署难度、提高了可扩展性,同时还确保了资源的有效利用和安全性。在微服务架构日益普及的今天,容器编排引擎已经成为了连接开发与运维、实现 DevOps 和持续集成/持续部署的关键,极大地提升了软件开发的效率和敏捷性^[18]。

随着容器化、微服务架构、声明式自动化以及持续交付等技术的进步和相互整合,行业内逐步建立起了一套全面的云原生环境体系。其通过在云端环境中构建和运行应用以充分利用云计算的弹性、可扩展性和敏捷性等优势,目前已经逐渐成为软件开发和运维的新标准。在云原生环境中,容器调度和镜像分发是两个关键技术,它们在保障应用部署效率和可靠性

方面扮演着至关重要的角色。

首先,在任务到来时,调度器需要在多个集群节点上动态分配容器实例。在实际应用中,不同任务的调度需求各不相同,如对某一特定资源需求较高^[19]、注重集群整体负载均衡^[20]、关注集群的资源利用率^[21]等,在不同的任务需求场景下,容器调度方式也需随之调整。目前容器编排引擎所提供的调度策略主要考虑容器的资源需求与限制,根据节点 CPU、内存的使用情况以及亲和性、反亲和性等因素进行调度决策。在实际场景中,默认的调度方式难以满足多种多样的任务调度需求,为此,用户可通过一系列接口规范进行拓展,针对场景需求设定自定义调度策略,从而提高容器运行的性能和效率^[22]。

其次,在启动容器前,节点需要先下载任务容器所需的镜像,因此镜像下载时延直接关系到容器启动的速度。在 Docker 架构中,镜像存储于一个远程的中心镜像仓库,当某节点需要启动容器时,会先从中心镜像仓库下载所需镜像到本地^[23]。研究表明,在使用 Docker 默认的镜像下载方式时,镜像下载时延占容器整体部署时延的 76%^[24]。此外在并发请求量较高时,多个节点需要同时从远程中心镜像仓库下载所需镜像,容易导致中心镜像仓库网络拥堵,从而增加了镜像下载时延。由此可见,镜像分发环节在容器生命周期中具有重要地位。因此制定合理的镜像分发策略,是提高容器启动速度的关键。

在实际应用中,提高容器部署速度能够提升集群整体的运行效率、缩短产品上市时间、进而提升服务质量。同时,企业常常遇到任务亟需快速启动的场景,例如在突如其来的流量高发时,需要迅速扩展服务规模以响应用户需求;在现有服务受到攻击时,需要尽快将补丁后的版本上线等。此类场景以可用性为最优先指标,更加注重于容器部署的速度。因此,研究云原生环境下容器部署加速策略具有重要的实际意义。

1.2 国内外研究现状分析

针对容器部署时所面临的难题,研究者从多个方面开展了研究,其优化方案主要聚焦于两个阶段:其一在容器调度阶段,关注如何根据任务场景需求选择最优的执行节点,确保应用的高效运行;其二在镜像分发阶段,关注如何更快、更高效地将镜像传输到部署环境中。本节将对这两个阶段的优化方案进行详细的分析。

在容器调度阶段,虽然容器编排引擎已经提供了一些常用的性能指标,但是在面对多种多样的任务场景时,仍然会有许多不足之处,因此有研究将性能指标进行拓展并与现有指标加权融合,从而适应不同的任务场景:

Zhang 等人^[25]提出了一种高效的自适应调度算法,综合考虑了运行容器主机的能量消耗、

镜像拉取的能量消耗和工作负载网络转换消耗成本,并通过线性规划计算得到最优调度方案。Liu B 等人^[26]提出了一种多目标的容器调度算法,考虑了节点 CPU、内存、镜像传输、集群节点相关性等因素,选取最优节点运行容器,有效提高了系统响应效率。Guo Y 等人^[27]提出了一种基于微服务邻域划分的容器调度策略,该策略在容器调度过程中考虑了系统负载平衡和系统响应时间这两个因素,有效地减少了系统负载失衡,提高了系统的整体性能。Menouer 等人^[28]使用 TOPSIS 多准则算法结合了分布式和集中式调度策略,使得调度结果在 CPU、内存、容器数量这三个指标上更加均衡。Guo W 等人^[29]在默认的负载均衡调度算法上,额外考虑了节点网络带宽利用率因素,并且通过 AHP 层次分析法更精确地设置了权重参数,该优化后的资源调度算法可以使集群的负载均衡性能更好。

此外,有相关研究通过使用高效的调度方式加快调度过程,或是选择性能较优的节点加速镜像分发:

常旭征等人^[30]针对 Kubernetes 调度器预选过程要遍历所有节点比较耗时的问题,提出了改进的资源调度算法,该算法在选出满足条件的节点数量时直接进行优选而无需遍历所有节点,从而提高了资源调度效率。在调度策略方面,Santos J 等人^[31]对 Kubernetes 进行网络感知调度扩展,在调度时评估的节点的网络负载情况;唐瑞^[32]提出了一种改进的 Kubernetes 资源模型,在 CPU、内存的基础上,将镜像网络下载速度和持久化存储数据传输速度作为资源调度的衡量因素。此类方法可以在将任务调度到网络情况较好的节点,使得镜像的下载更为迅速。Silvery Fu 等人^[33]提出了基于缓存依赖的调度策略,从减少下载量的角度提升容器部署速度。本文在调度算法上对其进行优化并设计了高效的调度框架,同时针对大规模任务场景下节点负载不均的问题,在原有方案的基础上进行了扩展。

在镜像分发阶段,现有研究主要聚焦于加速镜像传输和减少镜像下载量两个方面。具体方法包括 P2P 镜像传输、节点存储共享、镜像体积压缩以及镜像按需拉取等。

由于传统的镜像仓库下载方式常常因中心负载较高导致网络拥堵,现有研究常使用 P2P 传输技术加速镜像分发:

Wang Kangjin 等人^[34]设计了一个基于 P2P 的镜像分发系统 FID,该系统会为镜像的每个镜像层 Blob 制作种子文件,在节点拉取镜像时,先获取对应镜像的 manifest 文件,从中解析所需的各个镜像层,针对每一个镜像层,通过 BitTorrent 方式下载镜像层文件。Liang 等人^[35]设计了一种名为 HDID 的混合式镜像分发系统,该系统将 P2P 技术和从镜像仓库获取镜像的方式相结合,并根据镜像层大小,自适应地选择镜像仓库下载或是 P2P 传输,有效解决了 P2P 传输小镜像层效率较低的问题。张浩等人^[36]设计了一种异构云环境下高效的 Docker 镜像分发系统 RainbowD,该系统会动态地依据性能对集群节点进行排序,并在 P2P 镜像拉取时

优先选择性能较高的节点，从而在有限的资源下保证稳定、高效的镜像传输。贺龙^[37]提出了基于 Holt-Winters 模型的 Docker 镜像仓库负载预测方法，并设计了镜像分发模式决策策略。在传输镜像时根据仓库负载情况在默认下载方式和 P2P 传输方式中进行高效选择。姚增增^[38]同样采用了基于 BitTorrent 的镜像分发策略，但是借助了镜像仓库的 hook 机制，在 super node 上预先对所需镜像 Blob 制作种子文件，从而在收到某镜像的拉取请求时可以直接获取到种子文件，在一定程度上提高了下载速度。

Docker 采用非共享存储架构，每个节点会把完整镜像各自存储在本地。此方式没有考虑节点与节点之间的协作。因此有研究者通过集群节点间的存储共享与镜像协同管理，减少了镜像下载和存储的资源消耗：

Lian Du 等人^[39]设计了一种基于网络共享的 Docker 容器部署系统 Cider，该系统通过将工作节点的本地 Docker 存储更改为所有节点共享网络存储，同时将网络存储与本地文件级 COW 层相结合，在面对高并发部署作业时保障了系统的可扩展性和内存效率。S Nathan 等人^[40]设计了一种协同镜像管理系统 CoMiCon，该系统在多个节点之间以镜像层为单位分散镜像存储，单个节点只存储某个镜像的部分镜像层。在启动容器时，节点会分布式地拉取剩余的镜像层，从而高效利用了节点存储，提高了传输效率。Chao Zheng 等人^[41]对 Docker 非共享存储架构进行优化，设计了一种分布式镜像系统 Wharf。该系统使用分布式存储系统在集群中共享镜像文件，当节点需要下载镜像时，会优先检索并使用共享存储中的容器镜像并创建容器。当有多个节点需要同一镜像时，仅需在共享存储中保留一份镜像文件即可，从而减少了集群中同一镜像的副本数量，显著降低了网络和存储开销。

由于镜像的下载量直接影响着下载时延，因此有研究注重于对镜像本身的优化，通过删减不必要的镜像内容、使用比层级更小的细粒度或是优化镜像构建阶段的文件获取过程，从而有效减少了镜像下载量：

Lee H^[42]提出了一种新型镜像管理方法，通过设定通用核心组件共享依赖项，减少了基础镜像中的依赖冗余。周毅等人^[43]设计了一种本地存储与导出镜像优化工具 Docker-jimp，由于容器在运行时并不会访问到所有其中的文件，因此可对未访问的文件进行删减。该工具对一段时间内容器访问的文件进行保留，对未访问的部分，则对其未来是否会访问进行概率预测，删减低于阈值部分的镜像内容，从而减少了镜像体积的同时尽可能保证了文件完备性。Li H 等人^[44]设计了一种块级镜像服务系统 DADI，该系统在保留了镜像分层结构的同时，对镜像层进行分块，对于某一镜像层增删改的文件，通过 overlayBD 模块进行分层堆叠。在传输镜像时以更小的块级细粒度进行下载，从而减少了文件冗余。针对 Dockerfile 在构建不同镜像时，对于相同的文件获取指令会多次通过网络下载相同的文件，而无法复用本地已有缓存的

问题, Huang 等人^[45]设计了名为 FastBuild 的系统, 该系统将文件缓存功能无缝集成于 Docker, 从而透明地拦截和重定向对输入文件的请求, 减少了远程文件的重复下载。

现有框架在容器启动前会把所需的完整镜像拉取到本地。然而容器在启动时仅需镜像的部分内容, 剩余内容则只会在启动后的某个时刻访问到。因此可以在拉取镜像时先下载启动时所需要的部分镜像, 其他内容待需要时再下载, 从而错开网络高峰的同时加速容器的启动速度:

S Gotanda 等人^[46]设计了一种惰性层拉取的快速容器启动系统, 该系统在现有镜像结构的基础上, 额外增加一个仅包含容器启动时所需文件的镜像层, 并设计了一个名为 LlpFS 覆盖文件模块, 通过该模块, 节点只需下载镜像的启动层即可启动容器, 剩余的镜像层在后续按需下载。Simone Mosciatti 等人^[47]使用 CernFS 文件系统, 在 Kubernetes 平台上使用快照机制将镜像信息提供给容器运行时并启动容器, 其中镜像信息只包含了镜像元数据, 而不包括镜像文件本身, 只有在容器启动后才会真正去下载其所需的文件。Thalheim J 等人^[48]设计了一种轻量化容器构建系统 CNTR, 该系统将镜像文件中工具依赖部分和主应用程序部分进行分割。在启动容器时, 优先下载主应用程序部分, 而所需的工具依赖会在使用时再进行下载。卞盛伟^[49]设计了一种新型镜像结构 Gear, 在启动容器时只需下载很小的索引文件, 实际需要的文件将在容器启动后按需下载到本地。

Docker 在下载镜像时会并行下载镜像的各个镜像层, 其中镜像层以 gzip 格式压缩, 并在本地解压后构建为完整镜像, 有研究针对该过程的时延进行优化:

Arif Ahmed 等人^[50]从三个方面优化了镜像获取过程, 首先在镜像下载时, 以串行方式下载镜像层而非并行下载, 加快了第一镜像层的下载速度, 使得镜像解压过程更早到来。其次, 使用多线程的 pgzip 镜像压缩格式代替单线程的 gzip 压缩格式, 加速了镜像压缩文件的解压速度。最后, 在下载、解压和提取的过程中使用三个线程进行流水线式工作, 一旦某线程有数据可用, 便会传递到下一线程, 而无需等待某一流程结束。通过上述方法的互相结合, 从而提高了镜像获取速度。

1.3 本文研究内容及章节安排

本文研究的主要内容是云原生环境下容器部署加速策略, 从优化调度策略和加速镜像分发两个角度出发, 围绕减少镜像下载量、提升镜像传输速率、自适应调整容器部署方案等多个方面展开研究, 旨在解决节点缓存浪费、P2P 传输节点响应慢、可靠性不足以及整体部署流程缺乏灵活性的问题。具体内容包含以下三个方面:

1) 在容器调度阶段, **Kubernetes** 的本地存储优先策略仅考虑了完整镜像的复用, 忽略了不同镜像间可复用的镜像层, 导致了不必要的下载。针对此问题, 本文设计了一种基于节点镜像层缓存的容器调度方案, 通过将任务调度到已拥有所需镜像层最高的节点上以最大化利用节点缓存。在设计调度方案时, 本文充分考虑了镜像的层级结构特性, 减少了不必要的镜像层对比。在设计调度框架时, 本文预先将节点缓存信息存储于集群中, 并在任务到来前同步给调度器, 使得调度器可以更快获取节点缓存信息, 进而加快调度流程。实验结果表明, 本方案具有良好的调度效率, 并且在节点拥有缓存时能够有效减少镜像下载量, 提升容器部署速度。

2) 在镜像分发阶段, 现有方法通过 **P2P** 传输技术以缓解中心仓库的网络负载压力, 但 **P2P** 传输初始阶段源节点数量少, 传输效率较低, 因此难以适用于对服务响应速度有严格要求的任务场景。针对此问题, 本文提出了一种基于 **IPFS** 的分批式镜像分发方案。该方案首先使拥有所需镜像的节点构建最初的 **IPFS** 网络, 随后依据网络中实时的源节点数量, 分批次地将需求节点加入到上述 **IPFS** 网络中进行镜像传输直到所有需求节点获取到镜像。在设计镜像分发框架时, 本文通过相应组件保障了系统流程的自动化和高效性。实验结果表明, 相比于中心式镜像仓库下载和传统的 **IPFS** 传输方案, 本方案能够有效提升节点的平均响应速度并保障传输的鲁棒性。此外, 在节点规模较小时, 本方案可以有效减少镜像分发的总体时延。

3) 在实际应用中, 任务往往具有不同的部署需求, 同时集群的实时状态也会影响着容器的部署效率。因此本文在上述工作的基础上, 设计并实现一个基于 **Kubernetes** 的容器部署多阶段加速系统。该系统在任务到来前监测集群节点网络负载状况, 并在节点网络负载较低时预先构建节点缓存; 在任务到来时, 根据任务需求和集群实时状况, 灵活调整容器调度策略并自适应地选择最优的镜像分发方式; 在容器部署完成后, 为用户提供详细的日志以及统一的调优接口, 以便于对系统进一步优化。实验结果表明, 该系统可以在满足不同任务需求的前提下, 有效提升容器的部署速度。

综合以上研究内容, 本文主要分为六个章节。具体结构安排如下:

第一章阐述了现有生产环境下对容器相关技术的依赖, 分析容器部署加速的重要性和意义, 重点介绍了容器调度和镜像加速的相关研究现状和潜在价值, 分析现有方法的优势与不足。同时基于上述调研工作, 明确本文的主要研究内容和目标, 并对待解决的关键问题进行分析。最后介绍了本文的研究点与结构安排。

第二章详细介绍了本文所涉及的关键技术, 包括云原生技术背景、容器与镜像相关技术、目前广泛使用的容器编排引擎以及 **P2P** 传输。这些关键技术为后续研究内容奠定了坚实的基础, 并在本文中得到了深入探讨和应用。

第三章工作面向节点拥有一定镜像缓存的场景，提出了一种基于节点镜像层缓存的容器调度方案。首先分析了现有调度策略在容器部署速度上的不足，接着针对上述问题设计了高效的调度策略，随后通过 Kubernetes 自定义拓展的方式实现了相应的容器调度框架。在实验环节，通过与 Kubernetes 默认调度方案进行对比，验证了本方案的性能提升。

第四章工作面向对服务响应速度有所要求的并发任务场景，提出了一种基于 IPFS 的分批式镜像分发方案。首先对现有镜像分发方式进行对比与分析，选取轻量、高效的传输协议。接着在此基础上，分析其在该场景下存在的问题，并对原先的分发方式进行了优化。随后基于 Kubernetes 平台实现了相应的镜像分发框架。在实验环节，通过与中心式拉取和改进前的方案进行对比，验证了本方案的可行性和高效性。

第五章则在前两章的基础上，设计了一种云原生环境下基于 Kubernetes 的容器部署多阶段加速系统。首先分析了加速容器部署在实际应用中的重要意义，明确了本文系统的设计需求。接着对前文的工作进行完善并融合，将其构建为具体的工作模块，实现了一个具有较强灵活性和高效性的容器部署系统。最后通过实验验证了系统各模块所带来的性能提升。

第六章对本文研究工作进行了归纳总结，并对下一步工作进行展望。

第二章 相关技术综述

本章将对本文所涉及的相关技术进行详细阐述并分析。包括云原生技术、容器技术、主流的容器编排引擎以及 P2P 文件传输等。

2.1 云原生技术

云原生技术（Cloud Native）是一种为构建和运行可靠、高效、可扩展的应用程序而设计的技术体系。它致力于充分利用云计算环境的弹性、自动化和强大资源管理能力，以提供更高效的开发、部署和管理应用程序的方式^[51]。

传统的应用程序开发和部署方式往往面临着许多挑战，例如依赖于特定环境、难以扩展、缺乏弹性等。云原生技术通过将应用程序构建成容器的形式，充分利用容器的优势来解决这些问题。容器化可以将应用程序及其所需依赖项打包到一个独立的运行环境中，使其能够在任何云计算平台上以统一的方式运行。

此外，云原生倡导使用图 2.1 所示的微服务架构^[52-53]，其将单一应用程序划分成一组小的服务，每个服务运行独立的自己的进程中，服务之间互相协调、互相配合，为用户提供最终价值。这种松耦合的架构使开发团队能够更加敏捷地迭代和更新各个服务，提高开发效率和应用程序的可靠性。

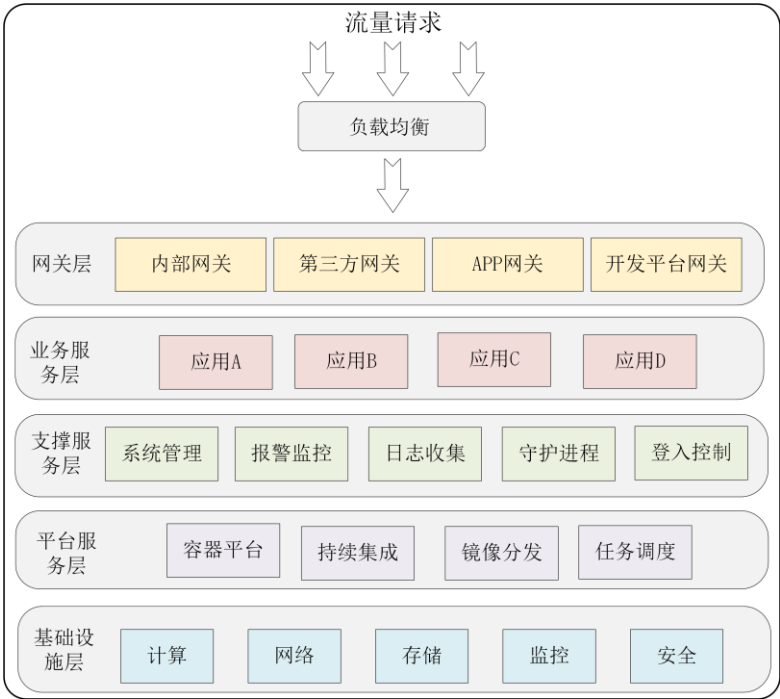


图 2.1 微服务架构

除了容器化和微服务架构，云原生技术还包括一系列的最佳实践和工具，用于自动化部署、弹性伸缩、服务发现和治理、监控和日志管理等方面。例如，通过容器编排平台可以有效地管理和编排容器化应用程序的部署，可以自动管理、调度容器并进行弹性伸缩，以及实现服务发现和负载均衡，使得应用程序可以在大规模的集群上高效运行^[54]。

同时，在应用开发的流程中，通过持续集成(Continuous Integration, CI)和持续部署(Continuous Delivery, CD)工具可以自动化构建、测试和部署应用程序，使团队能够频繁地发布新功能和修复 bug，同时保持高质量和稳定性^[55]，CI/CD 流程如图 2.2 所示。

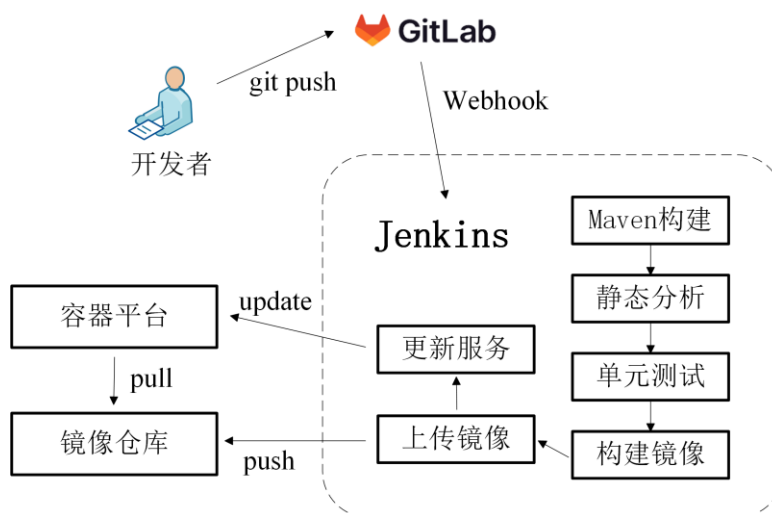


图 2.2 CI/CD 流程图

在云原生技术中，镜像分发是至关重要的一环。镜像仓库作为容器镜像的中央存储库，开发者可轻松上传构建好的镜像并进行版本控制和标签管理。在实际应用中，用户借助容器编排引擎和容器运行时，可以自动实现镜像拉取、分发和部署，极大简化了操作。专业工具如 Dragonfly^[56]提供了更多镜像分发功能，如 P2P 文件传输、隔离异常节点等，进一步提升分发的速度和可靠性。这些方法和工具共同确保了云原生环境中高效、稳定的镜像分发，为应用程序的快速部署提供了坚实基础。

总的来说，云原生技术正在改变软件开发和部署的方式。其通过使用容器化技术、微服务架构和自动化工具，提供了高效、可靠、可扩展的应用程序开发和运行方式。借助云原生的思维方式和相关技术，企业能够更好地满足新应用程序的需求，适应快速变化的场景并提高效率。

本文针对云原生背景下的容器调度和镜像分发两个领域展开研究，致力于容器部署加速策略的研究，旨在改善容器化应用的服务质量。本文系统设计基于微服务架构思想，所有组件均与原生组件解耦且可插拔。

2.2 容器技术

容器技术^[14]的出现可以追溯到操作系统层面的虚拟化,但与传统的虚拟机技术有所不同。在传统虚拟机中,每个应用程序运行在一个完整的虚拟操作系统上,而容器则共享宿主操作系统的内核,实现更高效的资源利用。容器将应用程序及其所有依赖项、运行时、系统工具和库打包在一个独立的单元中,从而提供一致的运行环境。在运行时,容器是一个相对隔离的单元,可确保一个容器中的应用程序不会影响到其他容器或宿主系统。二者的架构对比如图 2.3 所示。

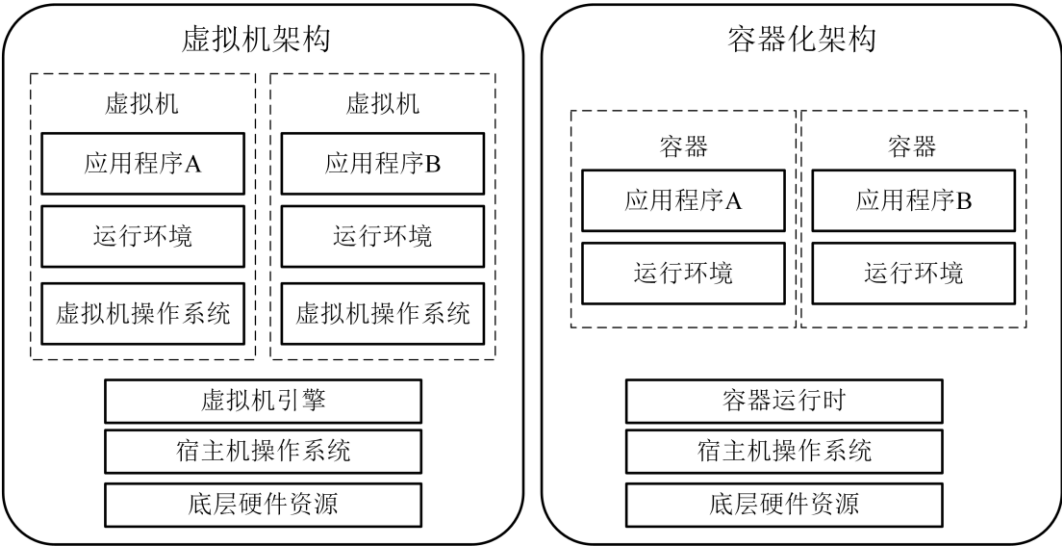


图 2.3 虚拟机架构与容器化架构对比

容器技术带来了多方面的优势,目前已成为现代软件开发的首选工具之一。相比于传统虚拟机,其优势主要体现在如下几个方面^[16]:

- (1) 轻量性。容器之间共享主机的操作系统内核,无需启动完整的操作系统,这使得容器启动更快,占用更少的系统资源。
- (2) 可移植性。容器可以在任何支持容器运行时的环境中运行,无论是开发环境、测试环境还是生产环境,简化了应用程序的部署和迁移过程。
- (3) 一致性。由于容器封装了应用程序本身及其运行时的依赖项,确保了在不同的环境中具有一致的运行行为,减少了由于环境差异性而引起的问题。
- (4) 扩展性。容器可以动态地调整资源分配,并根据负载的变化进行快速的水平扩展,更容易应对高流量或需求波动的应用程序。

其中,容器运行时是容器技术的核心,它提供了一套容器管理接口,用于创建、启动、停止和删除容器。通过这些接口,用户可以快捷地管理容器的生命周期。容器运行时还负责监控容器的状态,并提供日志记录和异常处理等功能,便于用户发现和解决错误。在众多容

器运行时中，Docker^[57]是最为流行和广泛使用的，它提供了简单而强大的命令行工具，使开发者能够方便地构建、运行和管理容器。容器运行时的实现依赖于操作系统提供的一些核心技术^[15]，主要包括：

（1）命名空间（Namespace）。Docker 利用 Linux 命名空间技术实现容器的隔离性。如表 2.1 所示，命名空间允许进程拥有独立的系统资源视图，从而实现进程、网络、文件系统挂载点、进程间通信、主机名以及用户 ID 的隔离。每个容器都运行在自己的命名空间内，确保了容器中运行的进程无法读取或影响到容器外的其他进程，从而使得容器内的应用与宿主机及其他容器相隔离。

表 2.1 Namespace 系统资源的隔离机制

Namespace 类型	系统调用参数	描述
Mount	CLONE_NEWNS	隔离各个进程的挂载点
IPC	CLONE_NEWIPC	隔离进程间通信
Network	CLONE_NEWN	隔离网络设备和网络资源
PID	CLONE_NEWPID	隔离进程的 ID
User	CLONE_NEWUSER	隔离用户和用户组的 ID
UTS	CLONE_NEWUTS	隔离主机名和域名信息

（2）控制组（Cgroup）。控制组用于限制、记录和隔离进程组使用的物理资源（如 CPU、内存、磁盘 I/O 等）。通过将进程划分到不同的控制组中，系统管理员可以控制每个组可以使用的资源量，从而实现资源的精确管理。在容器技术中，控制组成为了容器资源隔离和限制的关键，其确保了容器应用的资源使用不会超出预设，从而保障了系统的稳定性，提高了系统资源分配的灵活性和效率。

（3）联合文件系统（UnionFS）。联合文件系统能够叠加多个分离的文件层，将其构建为完整的文件系统视图。该技术支持文件的透明覆盖，可以在不影响原始文件的基础上，对文件进行修改和更新。在容器技术中，联合文件系统使得镜像以层的形式进行叠加，实现镜像的高效分发和存储，并简化了镜像的版本控制和快速回滚等过程。当容器启动时，联合文件系统将分散的层合并，为应用提供一致且完整的文件系统视图，从而确保容器在运行时能够高效地访问所需的文件和数据，无需关注底层文件的复杂管理。

2.2.1 容器镜像

容器镜像是一个轻量级、独立、可执行的软件包，包含应用程序的代码、运行时、系统工具、系统库和设置。在不同容器运行时中，镜像格式会有所不同，但基本都符合开放容器

接口^[58]（Open Container Initiative，OCI）格式，本文采用的容器运行时为 Docker，所述镜像均为 Docker 镜像。

图 2.4 展示了镜像的分层结构，Docker 使用层次文件系统来构建和管理镜像，允许将一个基础镜像与一系列上层更改组合起来。每个更改都是只读的，可以在上层继续添加更多的更改。这个机制可以高效地共享和重用镜像的组成部分，并且在构建和更新镜像时节省存储空间。容器在启动时使用了写时复制（copy-on-write，Cow）技术，其通过联合文件挂载技术在镜像的最顶层添加了一个读写层，所有运行容器时所做的更改（例如添加、删除、修改文件等）都会记录在这个读写层中。如果容器被删除，读写层也随之被删除，但只读的镜像层不会受到影响。

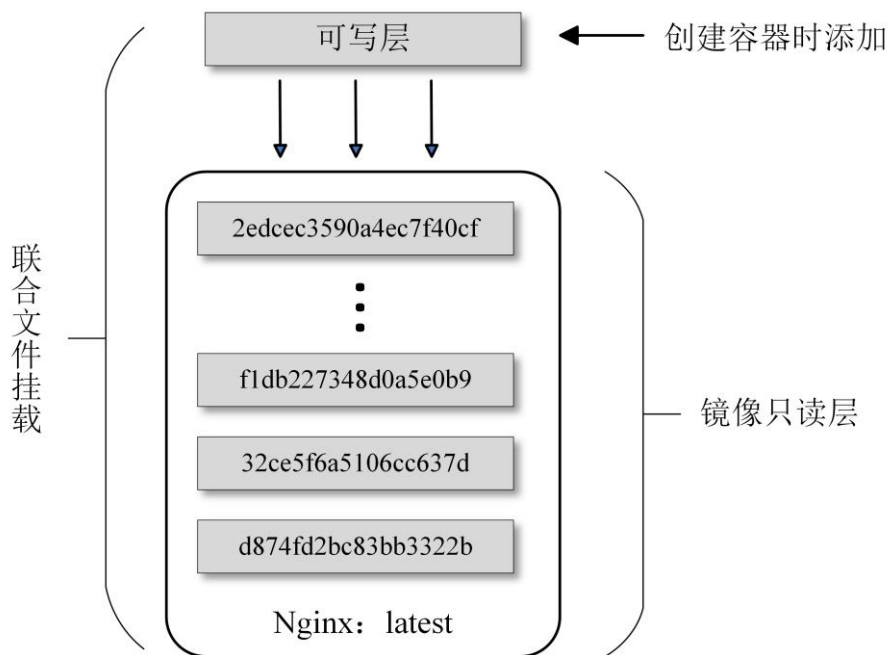


图 2.4 镜像分层结构

在构建 Docker 镜像时，需要先编写 Dockerfile 文本文件。如图 2.5 所示，其通过一系列指令，描述了镜像所需的基础镜像，并执行添加文件、安装软件包、配置环境变量和运行命令等操作。Dockerfile 使得镜像的构建过程可重复、可自动化，并且能够轻松地跟踪和管理镜像的变化。定义和构建 Docker 镜像的完整流程如下：

- 1、创建 Dockerfile 文件：首先需要创建一个名为 Dockerfile 的文本文件，该文件用于定义镜像构建过程中所需的指令和操作。
- 2、选择基础镜像：在 Dockerfile 中，通过 FROM 指令指定一个基础镜像，并从该基础镜像开始构建自定义镜像。基础镜像是一个已经存在的镜像，一般包含了操作系统和一些所需的基本软件。
- 3、定义镜像的构建过程：在 Dockerfile 中，通过使用一系列其他指令来定义镜像的构建

过程。例如使用 COPY 指令将本地文件复制到镜像中，使用 RUN 指令在镜像中执行 Linux 命令，使用 EXPOSE 指令声明此镜像应暴露的端口，使用 ENV 指令设置环境变量，使用 CMD 指令指定镜像启动后的执行程序等。

4、运行构建命令并生成镜像：在创建好 Dockerfile 后，使用 docker build 命令执行镜像构建。该命令会在当前目录下查找 Dockerfile 文件，并根据其中定义的指令逐步构建镜像。其中每条指令都会生成一个新的镜像层，并在上一层的基础上进行添加和更改。最后，Docker 将所有的镜像层打包在一起，生成一个完整的镜像。

```
FROM python:3.8-slim

WORKDIR /app

COPY . /app

RUN pip install --no-cache-dir -r requirements.txt

EXPOSE 5000

ENV NAME World

CMD ["python", "app.py"]
```

图 2.5 Dockerfile 文件示例

通过上述方法所构建的镜像将以层级结构存储于本地，用户可将其推送到远程镜像仓库，从而其他节点可以从镜像仓库中获取该镜像，后续将分析关于镜像分发的相关技术。

2.2.2 镜像分发

随着容器技术的不断推广，容器镜像的数目已达到数十万规模，随之而来的是镜像分发变得愈发复杂。在镜像分发的过程中，可能涉及不同的网络环境、安全需求和部署场景，其核心关注点主要在于以下几个部分：

（1）网络延迟和带宽限制：在容器镜像的传输时可能面临网络延迟和带宽限制。需要采取相关策略减少传输时间和资源占用。

（2）安全性：镜像分发的过程需要确保镜像的完整性和安全性，防止镜像在传输过程中被篡改或者恶意注入。

（3）版本控制：在分发过程中，需要有效地管理和跟踪不同版本的容器镜像。以便于应用进行滚动升级和回滚到先前版本。

为了应对上述问题，Docker 通过基于客户端——服务端（Client/Server，C/S）架构的镜

像仓库的方式存储、管理和分发镜像。在众多的镜像仓库中，Docker Hub 是一个公共的、默认的 Docker 镜像仓库，其包含了大量的官方和社区创建的 Docker 镜像。而对于一些组织或项目，可能需要使用私有的镜像仓库来存储和管理自己的镜像，从而更好地控制访问权限，并确保私有性及安全性。如图 2.6 所示，当镜像上传到镜像仓库时，会生成其对应的 manifest 文件，其中包含镜像的元数据信息、层的散列值等。当节点需求某一镜像时，Docker 会从镜像仓库中拉取所需镜像到本地，其中最直观的方式是采用 docker pull 命令，通过指定所需镜像的镜像名和版本号，Docker 会镜像仓库建立连接，先获取待下载镜像的 manifest 文件，从中解析所需下载的镜像层，并下载本地所缺乏的镜像层。

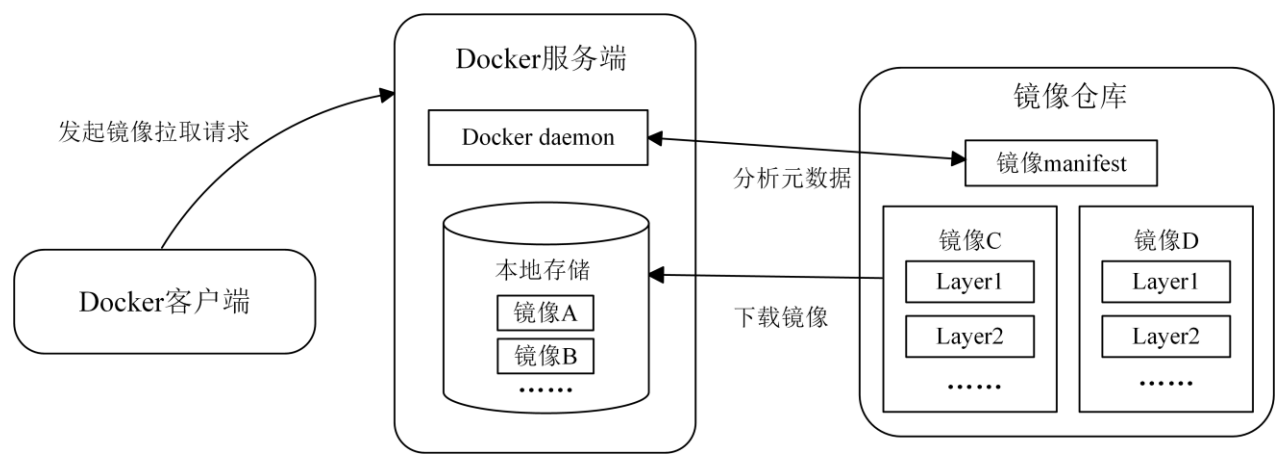


图 2.6 Docker 镜像拉取流程

由于 C/S 架构的中心化弊端^[59]，上述从中心镜像仓库获取镜像的方式在面对较多并发拉取请求时，可能会造成一定的集中网络负载，导致镜像拉取效率低下。为此有研究者采用点对点传输的方式，将镜像传输过程从中心式网络传输改为分布式网络传输，使需求节点从其他已拥有所需镜像的节点获取镜像，从而充分利用集群各节点的上下行带宽，提升镜像分发效率。

2.3 主流容器编排引擎

在当今云计算和微服务架构日益盛行的背景下，容器编排引擎的重要性不断凸显。容器编排引擎提供了自动化部署、扩展和管理容器化应用程序的能力，从而极大地提高了开发和运维效率。本节将深入分析几种主流的容器编排引擎，包括 Apache Mesos、Docker Swarm 和 Kubernetes，并比较它们的功能、优势及在实际应用中的适用场景。

2.3.1 Apache Mesos

Apache Mesos^[60]为管理数据中心或云环境中的计算资源提供了一个高效、灵活且可扩展

的部署平台。其总体架构如图 2.7 所示, 其中 Master 节点负责管理集群资源, Slave 节点负责执行任务, Scheduler 用于在应用层和管理层之间进行资源分配, Executor 负责运行诸如 Hadoop、MPI、Spark 等分布式应用程序。Mesos 最初是为了管理和调度大规模分布式系统中的资源而设计的, 但随着容器技术的兴起, Mesos 迅速适应了这一变化, 提供了对容器的原生支持。Mesos 的主要优势在于其高度可扩展性和强大的资源隔离能力, 是大数据处理和实时数据分析等资源密集型任务的理想选择。然而, Mesos 的系统架构和概念相对复杂, 且相较于其他解决方案, 其生态系统和社区支持较为有限, 导致其在部署配置和日常运维中具有较高的开销。

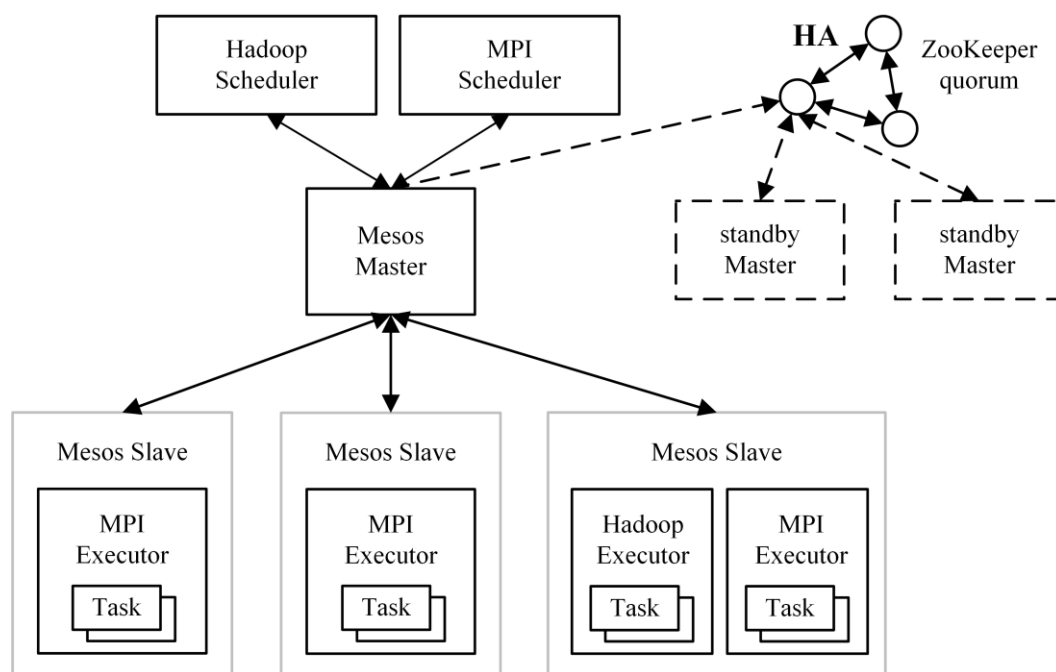


图 2.7 Apache Mesos 总体架构

2.3.2 Docker Swarm

Docker Swarm^[61-62]是 Docker 官方的容器编排工具, 旨在为开发者提供一个简易、快速的方式来部署和管理集群的容器化应用。其总体架构如图 2.8 所示, 管理节点(Swarm Manager)负责整个集群的管理工作, 包括状态维护、服务部署以及任务调度等。工作节点(Swarm Node)则负责运行容器实例, 执行任务。Docker Swarm 使用 Docker API 作为通信接口, 允许用户通过 Docker 命令行工具或其他兼容的客户端工具来管理集群, 有效降低了用户的学习成本。Docker Swarm 的主要优势在于简单、易于部署和使用, 非常适合中小型集群的部署场景。然而, 当涉及到更大规模集群管理和复杂应用部署时, Docker Swarm 会遇到功能瓶颈。

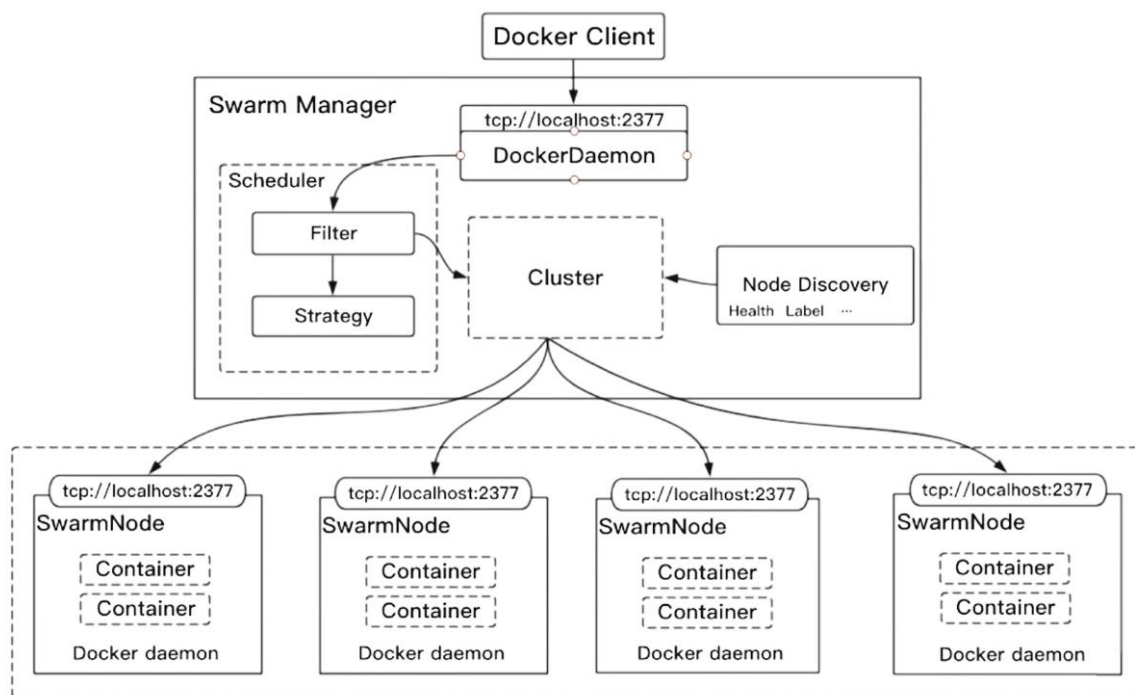


图 2.8 Docker Swarm 架构

2.3.3 Kubernetes

Kubernetes^[63]是由 Google 设计的开源容器编排引擎，旨在实现自动化的容器部署、扩展和管理，其总体架构如图 2.9 所示。Master 节点是 Kubernetes 集群的控制中心，负责管理整个集群，其中 API Server 是集群控制的入口，其他的所有组件都通过 API Server 进行协作；Controller Manager 负责监视集群中的资源状态，并根据用户定义的期望状态来自自动化部署、扩展和维护应用程序；Scheduler 负责将新创建的 Pod 调度到集群的工作节点上，其调度时会考虑 Pod 的资源需求、策略约束、亲和性和反亲和性等因素；Etcd 是一个分布式键值存储数据库，用于存储集群的配置信息、状态和元数据。Worker 节点是集群中的工作单元，负责运行应用容器，其中 Kubelet 用于与 Master 节点通信，负责监控 Pod 的运行状态；Kube Proxy 负责维护网络规则，使服务能够与集群内的其他组件通信。Kubernetes 拥有一个强大的生态系统，提供了一系列完备的功能，包括服务发现、负载均衡、自我修复和水平扩展等，同时支持多种容器运行时。此外 Kubernetes 拥有丰富的命令行工具和图形界面，使得应用部署、监控和管理变得更加简单。

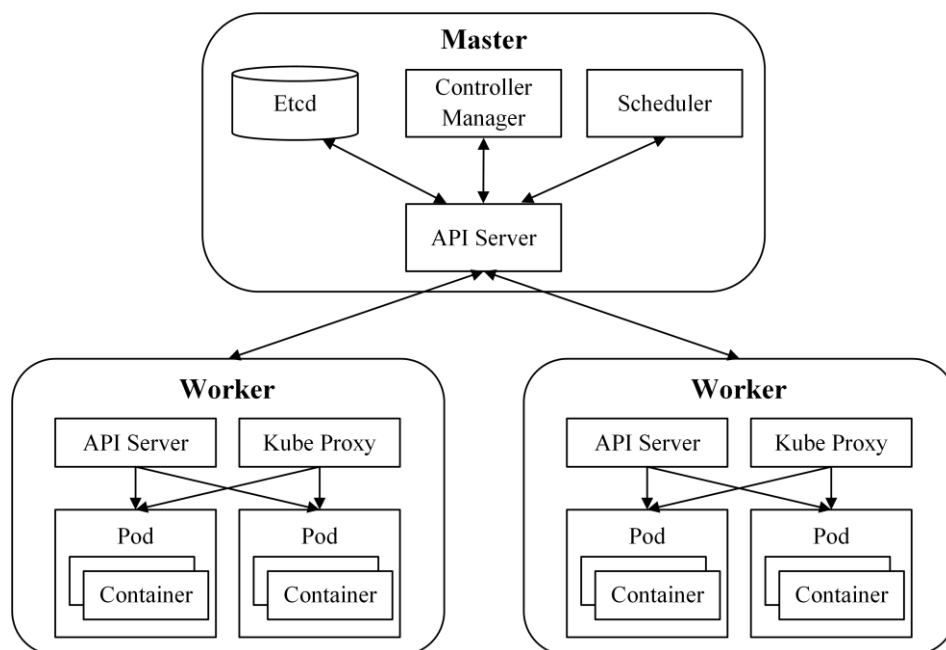


图 2.9 Kubernetes 系统架构图

对比其他容器编排引擎，Kubernetes 在社区支持、跨云平台兼容性、高度可配置性、自动化的运维能力等方面具有显著优势，目前已经成为当前市场上最受欢迎的容器编排解决方案。因此本文的系统设计基于 Kubernetes 平台，并对其架构下的容器部署策略进行优化。后续将论述 Kubernetes 资源对象相关技术。

Kubernetes 的资源对象是用于描述和定义集群中的各种计算、存储和网络资源的配置单元。其使用声明式配置模型，用户只需描述所需的系统状态，而不必明确指定如何实现。Kubernetes 控制器会负责按照期望状态来调整集群中的资源，从而为开发和运维人员提供了一种便捷的方式来描述和控制容器化应用程序的生命周期及其所需的基础设施。此类资源对象通过 YAML 或 JSON 文件进行定义，并由 Kubernetes API 进行创建、管理和监控。

本文主要涉及的资源对象包括 Pod、Deployment、DaemonSet、Job 和 ConfigMap。

Pod 是 Kubernetes 中最基本的部署单元，其中封装了一个或多个紧密相关的容器，这些容器可以共享网络命名空间、IP 地址和存储资源，从而优化了资源利用和通信效率。Pod 是资源对象的基础，几乎所有其他的资源对象都是围绕 Pod 设计的。

Deployment 允许用户定义期望的 Pod 的数量和配置，并由系统自动实现和维持这一状态。它支持应用的滚动更新、版本回滚、服务扩展和自我修复等功能，确保应用部署的可靠性和可用性。通过 Deployment，用户可以轻松管理无状态服务的多个副本，确保应用的持续部署、高可用和稳定运行。

DemonSet 用于确保在集群中的每个节点上都运行一个 Pod 的副本。常用于在整个集群中部署特定类型的系统级任务，如日志收集、监控、网络代理等。当有新节点加入集群时，

DaemonSet 会自动在这些节点上部署 Pod 副本，而当节点从集群中移除时，这些 Pod 也会被自动回收。

Job 用于在 Kubernetes 集群中执行短暂、一次性的任务，并在确保任务按预期执行完成后自动回收资源。任务执行完毕后，Job 会保留 Pod 的任务执行记录，便于用户访问 Pod 的输出、日志和最终状态，以进行故障排除或结果分析。

ConfigMap 用于存储应用的配置数据，允许用户在不修改容器镜像的前提下，通过环境变量、文件挂载等方式调整应用的服务配置。ConfigMap 的使用提升了微服务架构的解耦性，简化了应用部署和管理过程。

2.4 P2P 传输技术

P2P（Peer-to-Peer，点对点网络）是一种网络通信模型^[64]，如图 2.10 所示，其通过多个客户端直接连接实现资源共享，而无需经过中央服务器。在传统的客户端—服务器模型中，所有客户端的文件获取请求都会发送给中心服务器，由服务器统一提供下载服务，这在高并发时容易导致网络带宽竞争。而 P2P 网络中的每个节点既可以充当服务器，为其他节点提供服务 and 内容，又可以作为客户端，请求其他节点提供的资源，有效解决了集中式下载的网络负载瓶颈问题。此外，P2P 文件传输技术允许已下载文件的节点成为新的资源提供者，随着越来越多的节点拥有某一文件，原本集中的流量压力将被进一步分散。

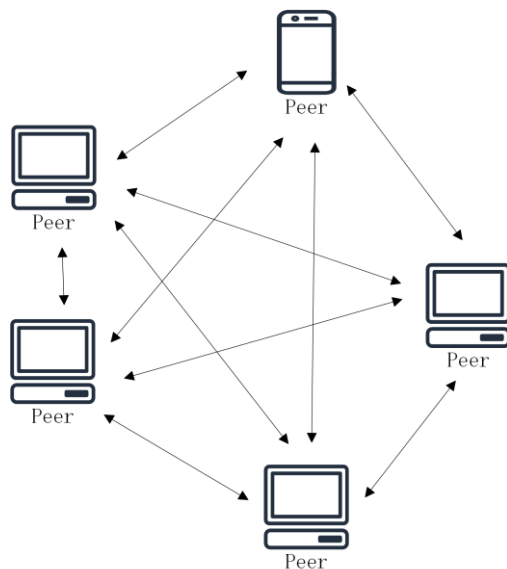


图 2.10 P2P 网络传输模型

2.4.1 比特流协议 BitTorrent

BitTorrent^[65]是一个在 P2P 网络中进行文件共享的传输协议，旨在提高文件的下载效率。

BitTorrent 协议引入了种子文件和 Tracker 服务器的概念。通过种子文件中包含的元数据，用户可以连接到一个集群网络，在其中寻找并下载所需的文件。而 Tracker 服务器则协调了不同用户之间的数据传输。此外，在 BitTorrent 网络中，文件被分割成多个小块，需求者可以从多个不同的节点并行下载不同的块，从而有效提升了下载速度。尤其是在节点数量较多的情况下，可供选择的下载源也随之增加，从而进一步加速了文件的传输速度。

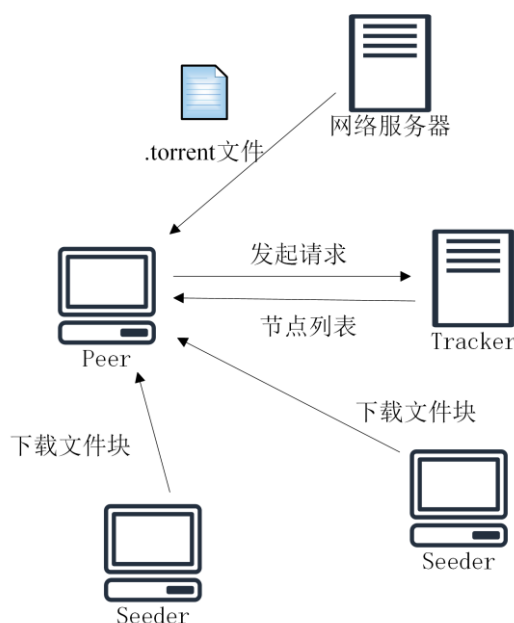


图 2.11 BitTorrent 协议架构

如图 2.11，其架构主要分为以下几个部分：

Peer： Peer 是指参与文件分享的节点。Peers 之间可以直接相互连接，从而实现节点间的文件传输。

Tracker： Tracker 是 BitTorrent 网络的中心化组件，用于协调各个用户之间的文件分享。它维护一个关于可用文件块的信息表，并向 Peer 提供其他 Peer 的信息。

Seeder： Seeder 是指已经完全下载文件并愿意分享的节点。该节点上存储了整个文件，而不仅仅是部分文件块。

在使用 P2P 进行文件传输时，首先一个节点（称为种子发布者）创建一个种子文件，其中包含有关文件的信息和 Tracker 的地址。随后下载者通过 Tracker 获取 Peers 的信息，Tracker 返回一个 Peers 列表，下载者可以从中选择要连接的 Peers。接着下载者从不同的 Peers 下载文件的不同块。在下载完成后，下载者可以分享文件给其他下载者。BitTorrent 使得大量用户能够共享文件，提升了文件下载速度。

2.4.2 星际文件系统 IPFS

星际文件系统^[66]（Inter Planetary File System，IPFS）作为 P2P 传输方式的另一种实现，致力于让 P2P 文件传输过程变得更加去中心化和分散化，其架构如图 2.12 所示。IPFS 利用分布式哈希表（DHT）构建节点之间的路由，使得网络中节点能够找到其他节点的位置。此外，IPFS 将文件内容转换成可寻址的形式，使得用户可以直接根据文件的内容来请求数据，而不必事先得知文件存储的地址信息。通过这一机制，用户在下载文件时无需种子或磁力链接，减少了传输开销以及对服务器的依赖。

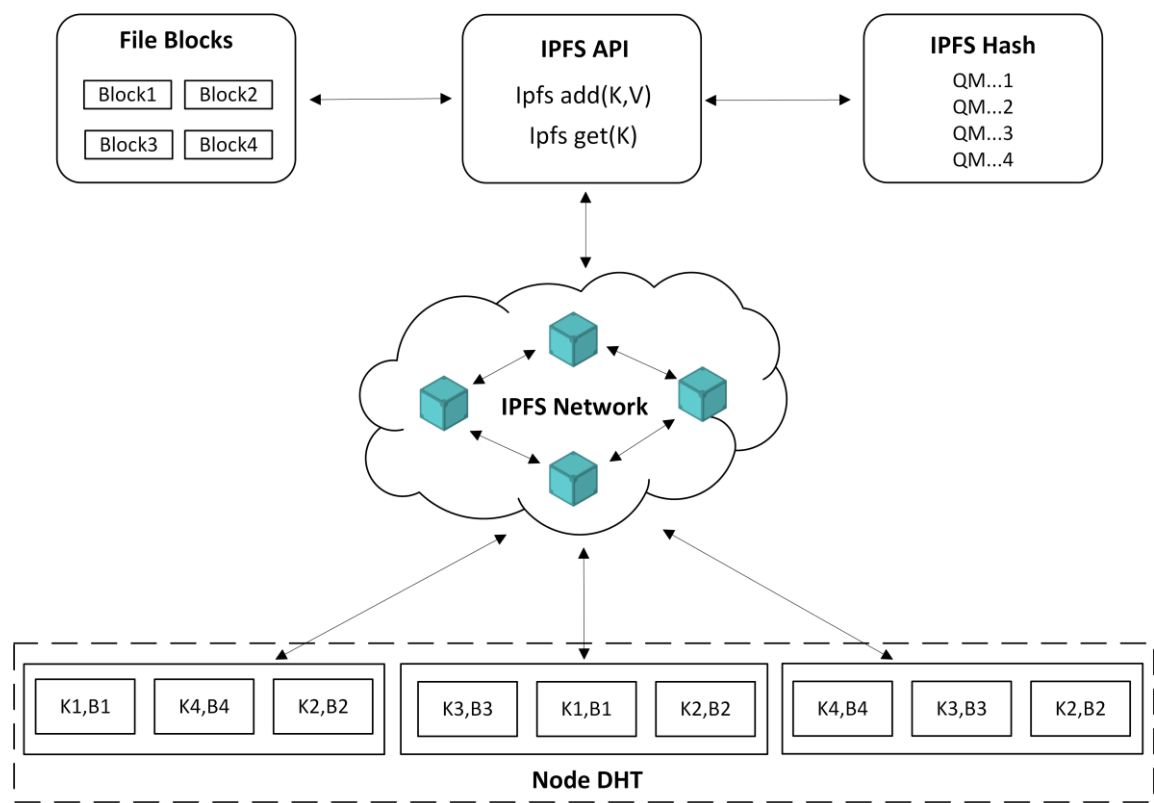


图 2.12 IPFS 架构图

当节点希望共享文件时，需要通过 add 方法将文件上传到 IPFS 网络中，此时文件会以多个块（Block）的形式存储，对于每一个文件块，IPFS 都会为其生成一个特定的 hash 值（如 QM...1）。在 IPFS 网络中，每个节点都维护了 DHT 的一部分，图中 K 代表了文件块的 hash，B 代表了文件块数据本身。当其他节点需求某文件时，需调用 get 方法并指定文件的 hash，通过 DHT 查找拥有各文件块的节点，随后在节点间建立连接并进行文件下载。IPFS 的轻量化设计使得文件传输过程更为简单高效。

2.5 本章小结

本章针对本文所涉及的相关技术进行了综述与分析，首先介绍了云原生相关技术与开发思想，接着讲述了容器相关技术，着重分析了镜像的分层结构与分发方式，然后对比了现有主流的容器编排引擎的优劣，并针对本文所基于的 **Kubernetes** 平台，对其相关技术原理、架构和资源对象进行了详细阐述，最后描述了 **P2P** 文件传输的相关技术背景。以上相关技术为本文后续的研究与实现提供了坚实的理论基础。

第三章 基于节点镜像层缓存的容器调度方案

本章设计并实现了一种自定义容器调度方案，该方案通过将待调度任务调度到镜像层缓存命中率最高的节点上，从而减少剩余镜像层的下载量，提升容器的启动速度。在应用层面，本章设计并实现了相应功能组件，并通过实验对比了性能优劣。

3.1 背景分析

近年来，随着容器技术的不断成熟和广泛应用，众多企业纷纷选择将业务迁移至容器云环境中，容器化应用的数量呈现爆炸式增长趋势，其复杂度和多样性也随之提升。由于不同类型的任务具有不同的调度需求，如高性能任务需要被调度到硬件配置较高的节点，大批量任务需要考虑整个集群的资源利用率以及负载均衡情况，突发任务则更侧重于部署的速度和服务可用性。因此在面对多种多样的任务需求时，制定合理的集群容器调度策略是保证服务质量的关键。

现有的调度策略主要聚焦于资源分配和负载均衡等方面^[67]，此类策略一是关注于任务的性能需求，从满足任务所需的硬件资源角度出发，注重待调度任务与节点性能的匹配^[68]。二是放眼于集群整体，考虑不同任务之间的差异，在合理分配有限资源的同时平衡负载^[69]。此类调度策略关注点在于资源本身，适用于特定需求任务以及提高资源分配效率的场景，忽略了对容器部署速度的性能优化，因此难以适用于任务亟需快速启动的应用场景。

表 3.1 列举了 Kubernetes 的默认调度器在调度任务时所包含的各项评分策略。其中大多数策略关注于节点资源使用情况和集群负载的平衡性，和部署速度相关的只有本地存储优先策略（ImageLocalityPriority）。该策略在一定程度上考虑了本地镜像缓存的复用，在调度任务时，会依据任务所需镜像的大小和节点的镜像缓存情况，偏向于将任务调度到所需镜像已存在于本地的节点。

表 3.1 Kubernetes 默认调度器评分策略

评分策略	说明
NodeResourcesBalancedAllocatin	根据节点的负载均衡情况评分
TaintToleration	根据节点污点容忍度评分
NodeResourcesFit	根据节点的资源已分配情况进行评分
PodTopologySpread	根据节点上运行的 Pod 数量进行评分
ImageLocalityPriority	根据镜像大小及节点缓存进行评分

在下载任务容器所需的镜像时,若本地已缓存有所需镜像的部分镜像层,则不会重复下载该部分镜像层文件。因此充分利用本地已有的缓存,可以有效减少下载量,提升镜像整体下载速度。在调度任务时,Kubernetes 的本地存储优先策略会获取集群中各节点缓存的镜像信息,若待调度任务所需镜像在某节点已经完整存在,则该任务会被更倾向于调度到该节点。此方法可以跳过镜像的下载阶段,极大提升容器的部署速度。而在实际应用场景中,待调度任务所需的镜像可能没有完整地存在于集群的某一节点,但是所需的部分镜像层却广泛存在于各节点上^[70]。其原因主要有以下几点:

(1) 镜像的底层是基础镜像,如操作系统镜像 centos, alpine, 编程语言环境镜像 Python, Java 等,而此类基础镜像往往被广泛使用,大概率已存在于集群中的部分节点。

(2) 镜像版本之间的升级迭代,其变化差异往往是微小的,体现为顶部的数个镜像层不同,而底部的数个镜像层则完全相同。因此拥有较低版本镜像的节点,往往缓存有大量高版本镜像所需的镜像层。

(3) 两个不同用途的镜像同样可能有着相同的基础镜像和环境配置,虽然在使用中二者并无直接联系,但在镜像结构中二者底层的数个镜像层是相同的。

在上述场景中,Kubernetes 的本地存储优先策略会因为节点没有完整所需镜像,忽略掉大量可复用的镜像层缓存。本文将针对该策略的不足之处,在细粒度上进行改进。后续将论述具体的算法与调度框架设计。

3.2 基于节点镜像层缓存的调度算法设计

基于上述分析,受文献[33]启发,本文设计了一种基于镜像层缓存的容器调度算法。该算法首先解析待调度任务所需的镜像层信息,随后计算各节点的镜像层缓存命中率并以此对节点进行评分,最终调度器依据评分结果选取缓存最多的节点执行任务以减少镜像下载量。在计算缓存命中时,本文考虑了镜像的结构特性,在最大程度上减少了算法的复杂度。以下是该算法的具体内容:

在收到待调度任务时,调度器根据任务的配置文件获取任务拟创建的容器信息;对于每个拟创建容器,获取其所需镜像的镜像名和版本号;根据镜像名和版本号从镜像仓库中获取该版本镜像的 manifest 文件;从 manifest 文件中提取该镜像的镜像层 digest 和 size;将所需镜像层的 digest 与节点缓存镜像层的 digest 进行对比,若相同,则将此镜像层 size 计入该节点的评分之中,若不同则跳过;待所有节点都进行评分后,对得分进行归一化处理;调度器选择得分最高的节点作为目标调度节点。节点得分的计算公式如下。

$$Score_i = PreScore_i \times MaxScore / MaxPreScore \quad (3-1)$$

式中, $Score_i$ 为节点 i 归一化后的最终得分。 $MaxScore$ 为设定的得分阈值区间, 通常设为固定值 100, 即节点最终得分的区间为 0~100。 $PreScore_i$ 表示对于此待调度任务, 节点 i 的镜像层缓存命中量, 其值为命中缓存的镜像层大小之和。 $MaxPreScore$ 表示对于此待调度任务, 单个节点的最大可能缓存命中量, 其值为待调度任务所需镜像层的大小之和。 $PreScore_i$ 与 $MaxPreScore$ 的具体公式如下。

$$PreScore_i = \sum_{l=0}^T MatchedLayerSize_l \quad (3-2)$$

$$MaxPreScore = \sum_{l=0}^T LayerSize_l \quad (3-3)$$

式中, T 为待调度任务所需镜像的镜像层层数。 $MatchedLayerSize_l$ 为节点 i 命中缓存的第 l 层镜像层大小。 $LayerSize_l$ 为待调度任务所需镜像的第 l 层镜像层大小。

其中, 在判断镜像层是否命中缓存时, 本文充分考虑了镜像的层级结构特性并提出了一种高效的镜像层匹配方式: 由于镜像的每个镜像层都是在前一层的基础上进行进一步修改得到的, 某一层的修改内容都会影响后续镜像层的 digest, 因此当所需镜像的某一镜像层未命中节点缓存时, 在其之上的各层也都不会命中缓存。于是在判断节点是否拥有所需镜像层时, 无需对所需镜像层和该节点上的层缓存进行逐一对比, 而是依据需求镜像的结构顺序, 优先对比底层的镜像层。若某一镜像层没有命中缓存, 则直接跳过剩余镜像层的对比, 从而有效减少了算法的复杂度, 提高了调度执行效率。

完整的节点评分算法如 Algorithm 1 所示。

Algorithm 1: Node scoring based on image layer cache

Input: node cache information

Output: node score

```

1: when receiving unscheduled task:
2: for each node in the cluster do
3:   get containers information of the task
4:   for each container do
5:     get digest and size of each image layer based on manifest
6:   end for
7:   sort the required image layers from lowest to highest
8:   for each required image layer  $l_1$ 
9:     for each cached image layer  $l_2$  do
10:      if digest of  $l_1$  == digest of  $l_2$  then
11:        calculate cache hits according to formula (3-2)
12:      else
```

```
14:          terminate cache hit detection on current node
15:      end if
16:  end for
17:  calculate maximum possible cache hits according to formula (3-3)
18: end for
19: calculate final score according to formula (3-1)
20: end for
```

调度算法总体流程如图 3.1 所示。本算法最终归一化后的得分表示了某节点所缓存的镜像层大小占所需镜像层大小的百分比，能够直观地表示缓存的命中率。若有多个节点获得了相同的最高得分，调度器则会在其中随机选取目标节点。

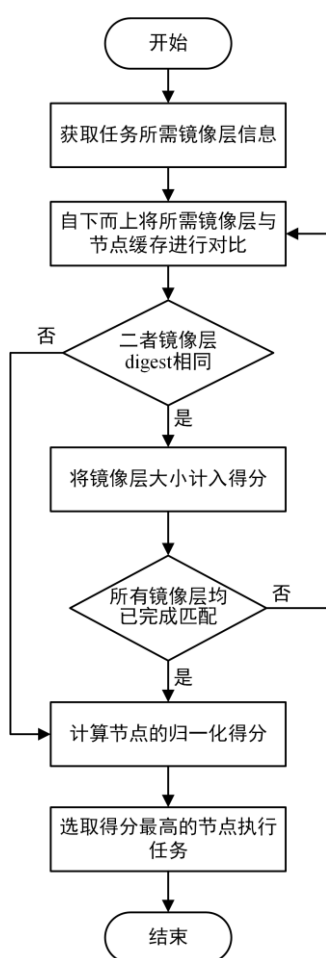


图 3.1 基于节点镜像层缓存的调度算法流程

3.3 基于节点镜像层缓存的调度框架设计

本调度方案的核心目标是将任务调度到镜像层缓存命中率最高的节点上。为实现此目的，本节设计了如图 3.2 所示的调度流程。首先，需要收集节点镜像层缓存信息并将其提供给后续组件。随后，需要设计相应调度器，根据用户所创建的任务，依据评分算法对节点进行打

分,接着选取得分最高的节点作为目标调度节点,将任务调度到该节点上。此外在任务调度完成后,需要及时更新节点的镜像层缓存信息。

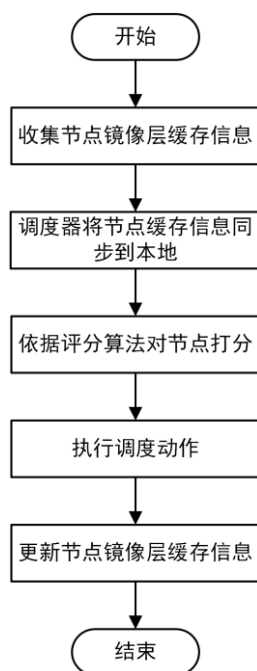


图 3.2 调度框架流程设计

本节后续将针对上述流程设计具体的系统组件。考虑到此调度方案主要面向于快速启动场景,需要尽可能地提高组件的运作效率,因此调度器在收集各节点缓存的镜像层信息时,采用了预先缓存而非即时查询的方式,从而在待调度任务来临时,能够更快调用上述信息,提高组件运作效率。

为降低配置难度、便于统一规范管理,本文的调度框架基于广泛使用的 Kubernetes 平台,借助 Kubernetes 现有功能并基于本文的调度算法对其进行拓展。同时选用 Docker 作为底层容器运行时管理容器的生命周期。

在实际应用场景中, Kubernetes 默认的资源对象和调度策略可能无法满足各式各样的任务需求,为此, Kubernetes 提供了一系列扩展机制供用户根据实际需求对平台进行调整和扩充,从而更好地满足各种不同的业务场景。基于上述机制,本文通过自定义资源定义 (Custom Resource Definition, CRD)、自定义资源 (Custom Resource, CR)、控制器 (Controller) 和调度器 (Scheduler) 扩展,设计了一个基于节点镜像层缓存的容器调度框架,其主要包括以下三个核心组件:

(1) 信息存储组件 (Layer-storage)。该组件通过 Kubernetes 自定义资源定义实现,用于存储集群中各节点所缓存的镜像层信息。该信息可快速调用并通过控制器保持更新。

(2) 缓存更新组件 (Layer-updater)。该组件通过 Kubernetes 自定义控制器实现,用于为各节点生成对应的自定义资源、根据节点所存储镜像的镜像名,获取并记录对应的镜像层

（3）评分调度组件（Layer-scheduler）。该组件通过 Kubernetes 自定义调度器实现，用于执行本章 3.2 节所提出的调度算法。该组件在调度前预先将节点缓存信息同步到本地，在调度任务来临时依据评分算法对节点打分，选取评分最高的节点作为任务执行节点。随后，该组件通过 API Server 与目标节点的 Kubelet 组件进行交互，并通知该节点的 Docker 容器运行时下载镜像，完成后续的容器启动任务。

框架整体架构如图 3.3 所示，其中 Worker1 为示例的最终调度节点。后续将详细论述各组件的具体实现。

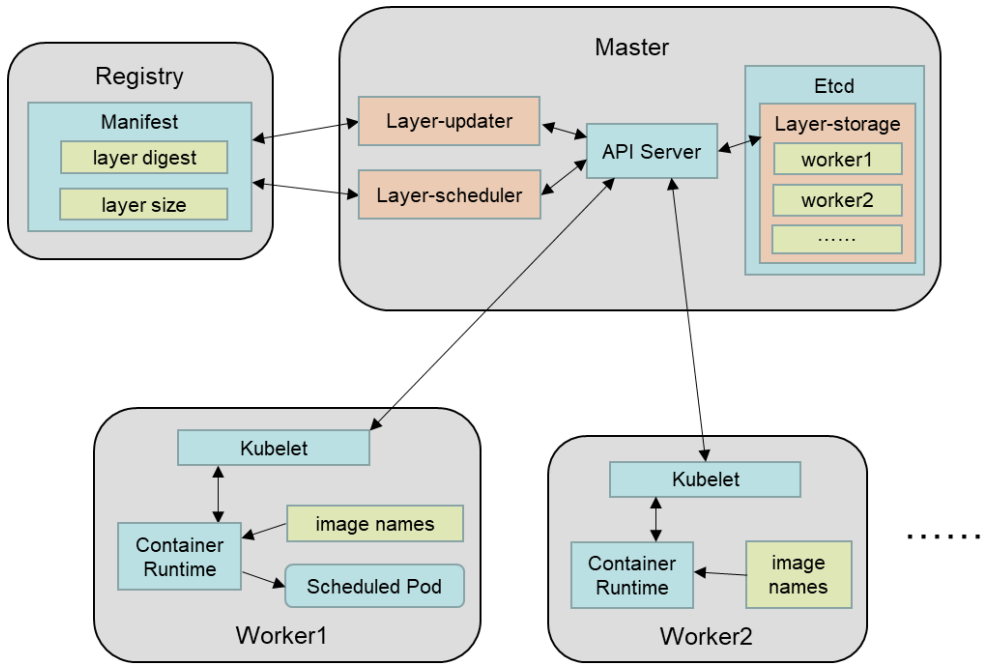


图 3.3 基于镜像层缓存的容器调度框架

3.3.1 信息存储组件

该组件基于 CRD 实现,CRD 是 Kubernetes 中用于创建用户自定义资源类型的 API 对象，详细规定了自定义资源的数据模型，包括其字段名、数据类型等，从而为自定义资源的创建、存储、管理提供了一套规范化框架。在 CRD 注册到 Kubernetes 集群后，集群的 API 服务器将能够识别并正确处理与该自定义资源相关的 API 请求。

该组件将各节点的缓存信息抽象为集群中的一种资源类型，便于调度器进行调用以作为节点评分的依据。Kubernetes 通过资源对象的 status 字段来表示其在系统上的当前状态，本文在设计自定义资源类型时,在其 status 字段中添加了节点名(Node Name)和层信息(Layers)子字段以表示某个节点当前拥有的镜像层信息。如表 3.2 所示，Node Name 字段为字符串类型，记录了节点的名称。Layers 字段为键值对类型，其中键记录了镜像层的 digest 值，标识

表 3.2 CRD 字段定义

字段名	类型	含义	
Node Name	字符串	节点名	
Layers	键值对	键：镜像层 digest	值：镜像层 size

3.3.2 缓存更新组件

该组件基于自定义 Controller 实现，在 Kubernetes 中，Controller 组件是实现控制器模式的核心，其负责通过 API Server 提供的 List & Watch 接口实时监控集群中资源对象的状态变化，确保系统的稳定运行。如图 3.4 所示，控制器首先使用 List 操作来获取资源的初始状态，随后通过 Watch 操作获取这些资源的更新事件，从而无需进行周期性的轮询就能实时地对资源状态的变化做出响应，保持集群的实际状态与期望状态的同步。

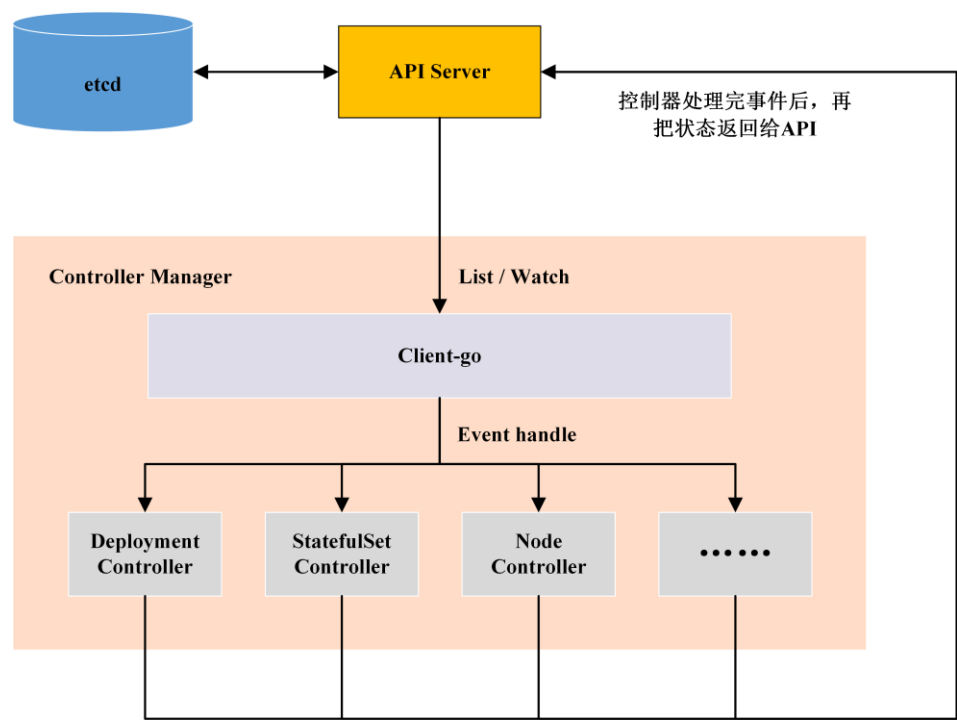


图 3.4 Kubernetes 控制器工作模式

控制器的工作流程如下：

- （1）观察 (Observe):从 API Server 读取资源对象的期望状态，并持续监听资源对象的资源状态变化。
- （2）比较 (Compare): 将观察到的状态与期望状态进行比较，判断二者是否存在差异。
- （3）调谐 (Reconcile): 如果检测到实际状态存在差异，控制器会采取相关措施来调谐系

统状态，使其与期望状态一致。

在执行调谐后，控制器会更新资源对象的状态字段，以使用户和其他控制器可以查询并了解系统中发生的变化。控制器是一个长期运行的进程，不断重复观察、比较、调谐和更新的过程，确保系统状态始终符合期望状态，使得系统状态的管理更为简单、高效。

由于节点信息存储组件通过自定义资源记录了节点的镜像层缓存情况，而 Kubernetes 原生控制器只能管理其内置的资源对象，当涉及到监视和响应自定义资源的状态变化时，这些原生控制器则无法直接使用。为了保持其信息与集群中各节点的实时状态同步，本文通过自定义控制器设计了缓存更新组件。该组件负责实时收集各节点的镜像层缓存信息，并将这些信息更新到各节点 CR 的 status 字段中。由于在 Kubernetes 集群中，某节点缓存的镜像层取决于该节点运行过的 Pod，因此本文设计的控制器采用事件监听触发机制更新 CR 对象，在 Kubernetes 集群中发生 Pod 的创建与修改事件时，进行 CR 对象的字段更新操作。节点 CR 对象示例如图 3.5 所示，其对应了表 3.2 中所列举的需要记录的节点信息。

```
status:
  Layers:
    2e254547242d2d1ee5687eda61fa0f679a50e663596cd5767cc9af8427d69688: "841999"
    53a43e5eee08a5ec0ab8e3663f9ef85bf9a99b620d49e0b164814eca02ce4e3f: "1490"
    504c1e01744e6253d30c6d328e736699ccb503efb2fd0fa9abf312ed9e269f5a: "41373899"
    5e8995dba7152ca33817298eb9c454a8e95bddf717d978516034c2f53508405f: "957"
    74a4f808141d600caab8fa8745ba05c6fcb554953ae887be7de6df183bc3528a: "1210"
    86ca2e851f883750c31c83d5ffe44e067a0cc3ca56c61713c52cb4b6e862e285: "1079286"
    330fd09f430673a53d707428068c13fb0001fc784e89d505bbd968d1b81a1f7d: "1400"
    7772eadea24115ce605abcada8ee3074786e54e6520cfd2b4184bacc837e1eeb: "5433642"
    a1330b43d726d24177d547648c93c5b8fc0204b929f895e428c153a62ca45a1e: "629"
    b790c763077d39e4465543261cf1fb8f2a4f5c60219402b3854c8c8a57dff868: "3412768"
    c9e5d24ac9dfa67baf9c184bf6fce5f74273222355b8a581de6cf972f47848e0: "5966601"
    cee2516f7556965a20a1729b7462c6653130a9678459cd47168784c45e65682a: "1203"
    d5181593591ea7cf6ae1559e5caf24944b685b94ef9d32addfd80db069c75d35: "367"
    db6f5e13c16495a4d399cfaee50e3ed8c08d47f5f92be1b4f3dbae6558d141f4: "833027"
    dbb559d487e116bd397eda1da966517bbc666d0a2b1fce29af5643be371a0751: "833195"
    e1caac4eb9d2ec24aa3618e5992208321a92492aef5fef5eb9e470895f771c56: "29124091"
    fee0a83203ad65d3974dba6eae37f03a7aa3344a69b8b631991a5c8a61e77b04: "12261202"
  NodeName: worker2
```

图 3.5 节点 CR 对象示例

此组件架构如图 3.6 所示，工作流程如下：①根据 Kubernetes 集群中每个节点名称，判断是否已经存在名称等同于节点名称的 CR 实例对象，若不存在则先进行创建以保证集群中每个节点都有各自的 CR 实例对象。②通过 API Server 与 Kubelet 交互获取各节点所缓存镜像的镜像名及版本号。③根据镜像名和版本号，从镜像仓库先获取该镜像的 manifest 文件，再从 manifest 文件中提取该镜像各层的 digest 和 size 信息。④对该节点上所有镜像重复此操作后，将其以键值对格式记录在该节点 CR 实例对象的 statue 字段中。后续当集群中发生 Pod 的增改事件时，由于已存在每个节点的 CR 实例对象，只需从流程②开始执行即可。

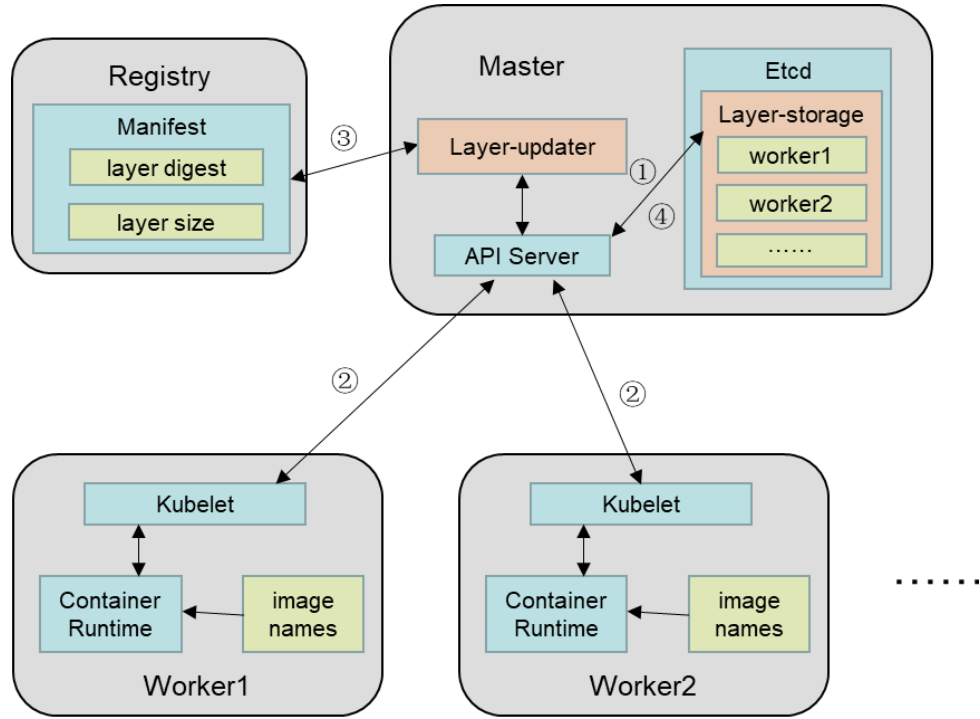


图 3.6 缓存更新组件架构

由于在 Kubernetes 集群中，各节点上都缓存了用于构建 Kubernetes 集群系统的镜像，此类镜像没有复用的可能，因此该组件在获取镜像名时会过滤掉该类型镜像。另外，对于某一特定镜像，从镜像仓库中获取 manifest 文件的操作是一次性的，在首次获取 manifest 文件后，该组件将镜像名与镜像层信息的对应关系进行缓存，若后续有相同镜像需要获取层信息则直接从记录中调用，从而避免重复获取带来的网络开销。

本文没有直接从各节点本地文件存储中获取镜像层信息^[33]，其原因在于镜像的 manifest 文件对应了 Registry 服务端的镜像配置。在下载镜像时，镜像层通常以 gzip 压缩格式下载到本地后再进行解压，解压前后镜像层的 digest 值不同。因此在 manifest 文件中获取到的镜像层 digest 与本地存储的不同，二者通过构建 digest 对应关系的文件以关联解压前后的镜像层。本文依据镜像名和版本号获取镜像 manifest 文件从而记录镜像层信息的方式，相比于该方法具有一定效率提升：首先无需查询上述对应解压前后 digest 的文件。其次镜像的下载虽然是以镜像层为单位，但是不会单独出现某个独立的镜像层，因此以镜像为单位统计无需频繁访问本地文件系统，过程更加高效。此外，本文记录镜像层 digest 只服务于调度时的镜像层缓存对比，因此无需对每个具体的镜像层计算 digest，只需对比解压前二者的 digest 是否相同即可，从而避免了不必要的计算。

3.3.3 评分调度组件

该组件基于自定义 Scheduler 实现，Kubernetes 的 Scheduler 组件负责决定容器在集群中的哪个节点上运行。Scheduler 的主要目标是考虑任务的资源约束和污点容忍，同时优化资源利用，确保集群中的工作负载能够在可用的节点上平衡分布等。其工作流程如图 3.7 所示。

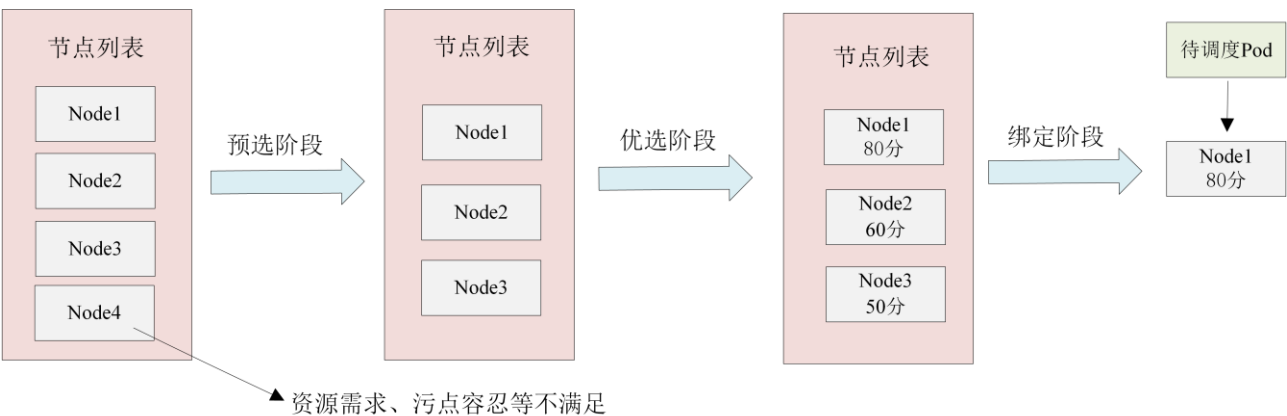


图 3.7 Kubernetes 调度流程

其主要包括以下步骤：

- 1、任务提交：用户指定需要创建的 Pod 并将其提交到 Kubernetes 集群。Pod 中包含有关应用程序容器和调度需求的规范。
- 2、节点预选：当 Pod 被提交时，Scheduler 会根据一系列规则和算法进行节点预选，包括考虑资源需求、污点容忍规则等。满足预选条件的节点进入优选阶段。
- 3、节点优选：对于通过预选的节点，Scheduler 会对其进行更详细的评分，主要包括节点 CPU、内存的负载情况等。调度从集群可用节点中选择评分最高的节点作为运行该 Pod 的节点。
- 4、Pod 绑定：Scheduler 将 Pod 与最终决策的节点绑定，确保 Pod 在集群中的最佳节点运行。绑定完成后，该节点的 Kubelet 会收到关于新 Pod 的信息，并与容器运行时交互，开始拉取 Pod 所需的容器镜像，最终在该节点上启动 Pod。

默认情况下，Kubernetes 使用内置的调度器进行 Pod 的调度，但本文需要根据节点镜像层缓存这一特定需求进行调度，因此本文借助了 Kubernetes 提供了调度器框架（Scheduler Framework），如图 3.8 所示，该框架允许用户设计自定义的调度器，在不同的扩展点实现调度算法和逻辑。

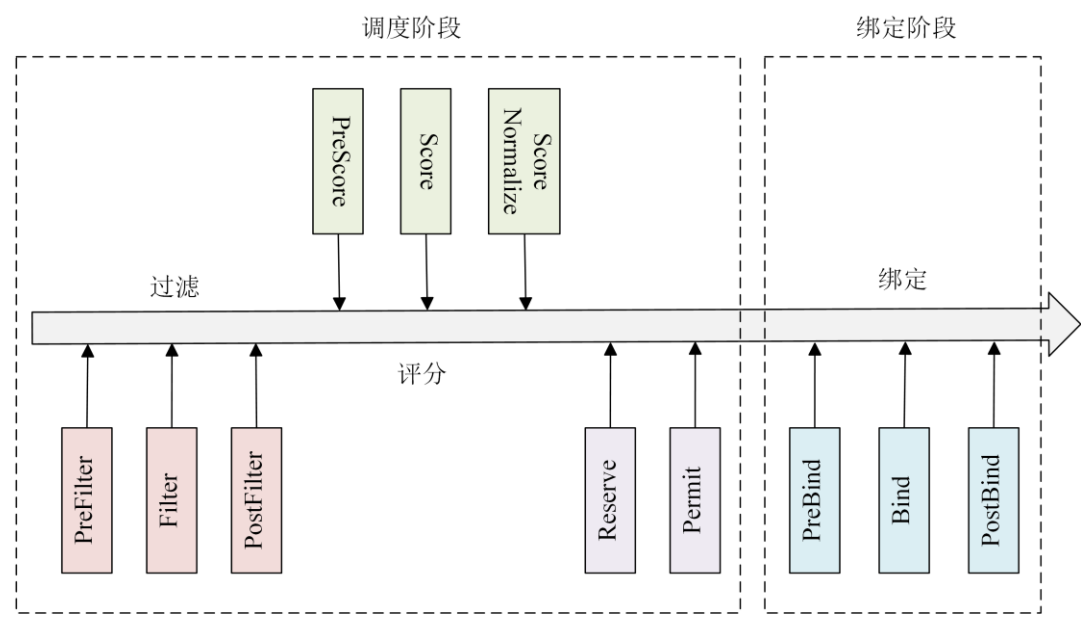


图 3.8 Kubernetes 调度扩展点

基于上述分析，本文通过自定义 Score 扩展点，设计了依据节点本地镜像层缓存，将待调度 Pod 调度到缓存命中率最高的节点上的调度器。此组件架构如图 3.9 所示。组件流程如下：

- ①在任务到来前，调度器读取各节点 CR 信息并将其存储在本地。
- ②在调度任务到来时，调度器根据用户创建的任务，获取待调度 Pod 中各容器所需镜像名及版本号。
- ③调度器根据镜像名及版本号从镜像仓库获取 Pod 所需镜像的 manifest 文件，得到镜像层 digest。并依据各节点 CR 对象中的镜像层信息对各节点进行打分。
- ④调度器将待调度 Pod 与得分最高的节点绑定。
- ⑤该节点的 Kubelet 与容器运行时交互，进行 Pod 创建并下载剩余所需的镜像层、启动容器。调度时具体的评分算法详见 3.2 节。

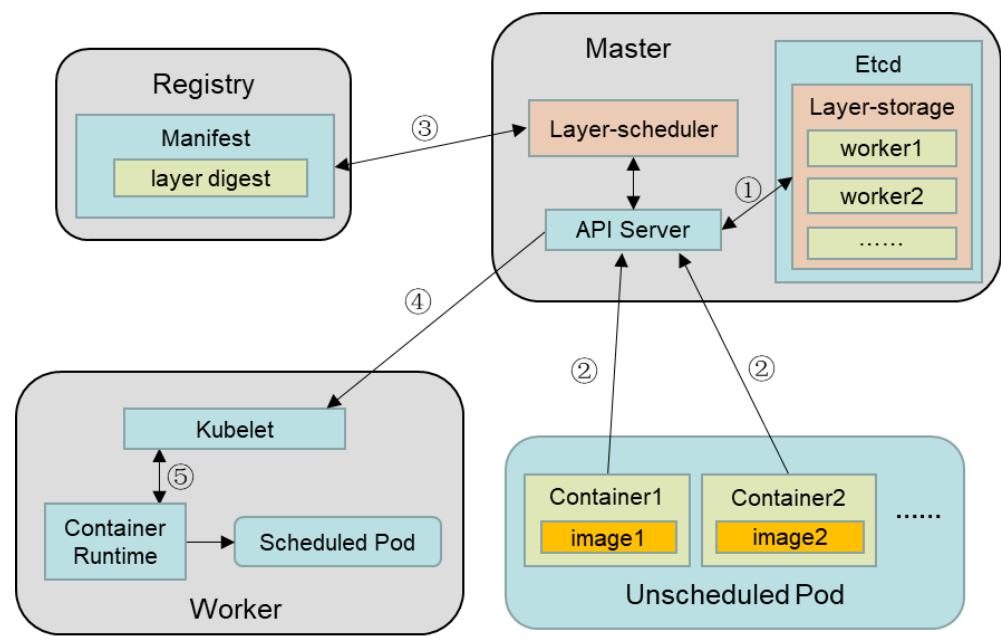


图 3.9 评分调度组件架构

在架构设计上,对 Kubelet 进行扩展从而实时读取镜像层缓存同样是一种可行方案^[33],但相比于本文方案,其存在调度效率较低的问题。本文方案将集群中各节点镜像层缓存信息记录于 CR 对象中,并在调度任务来临前提供给调度器。在调度任务来临时,调度器只需直接读取本地缓存便可得知各节点上的缓存信息,该缓存信息在调度任务来临之前获取,并且在调度任务完成后进行更新,因此不会带来额外的调度时长开销。而通过 Kubelet 扩展获取镜像层缓存信息的方式,需要调度器在调度时与 API Server 进行实时交互,从而带来了额外的调度时延。尤其在大规模集群场景下,节点和镜像数量繁多,交互时延将显著提升。此外,在部署复杂度方面,拓展后的 Kubelet 需要重新部署在集群中的每个节点,而本方案只需在主节点上部署上述组件即可,在架构上更具轻量性。

3.4 实验与分析

为验证此调度方案在实际应用中的表现情况,本文设置了三组不同缓存命中场景的对照实验,分别为:

- (1) 集群中所有节点都不存在调度任务所需的任何镜像层,即预期缓存命中率为 0%;
- (2) 部分节点拥有待调度任务所需的部分镜像层,即预期缓存命中率介于 0%~100%。
- (3) 部分节点拥有待调度任务所需的完整镜像,即预期缓存命中率达到 100%;

本文首先进行功能性验证,通过设定相应的缓存配置,记录在上述情况下各节点的最终得分以及调度结果,从而验证本系统功能的有效性。随后对比本文调度系统和 Kubernetes 默认调度器在上述场景下的容器部署速度,从而验证本系统功能的高效性。其中场景二在实际生产环境中最常见,是本系统的主要应用场景,本文针对此场景,逐一模拟 3.1 节所述的三种部分镜像层缓存命中的情形,以验证实际应用中的性能提升。

3.4.1 实验环境

本实验使用的 Kubernetes 版本为 1.19,底层容器运行时选用 Docker 19.03。集群中共有 4 个节点,以虚拟机的形式部署在同一宿主机上,各节点配置如表 3.3 所示。其中 master 作为主控节点,worker 节点作为任务待调度节点,调度框架的各模块均以插件化的形式从 master 节点部署到集群中。

表 3.3 实验环境

节点名称	系统	CPU 核数	内存 (GB)	带宽 (Mbps)
master	Centos7	4	6	1000
worker1	Centos7	2	4	1000
worker2	Centos7	2	4	1000
worker3	Centos7	2	4	1000

3.4.2 镜像层缓存未命中场景

该场景下所有节点都未拥有所需缓存，容器为完全冷启动。本文采用 mysql 镜像作为测试镜像部署容器，该镜像大小为 175MB。任务以单个 Pod 的形式部署到 Kubernetes 集群中，各 worker 节点缓存、待调度任务及调度结果如下表 3.4 所示。

表 3.4 节点缓存、得分，待调度任务及调度结果

节点名/待调度 Pod 名	缓存镜像/所需镜像	节点得分/调度结果
worker1	无	0
worker2	无	0
worker3	无	0
test1	mysql:latest	worker1

实验结果表明，所有 worker 节点均无镜像层缓存命中，所获得分均为 0，调度器随机选择可用节点进行调度。默认调度器的实验结果同样为随机选取。在时延方面，此情形下本方案无法发挥其优势，同时在调度时需要获取待调度任务所需镜像的 manifest 文件，因此需要额外的调度时间，在容器部署速度上不及默认调度器。

3.4.3 镜像层缓存部分命中场景

对于该场景，本文根据 3.1 节模拟以下三种情形：

其一，待调度 Pod 所需镜像由某基础镜像构建而来，该基础镜像已存在于部分节点。

其二，待调度 Pod 所需镜像为某节点拥有镜像的不同版本。

其三，待调度 Pod 所需的部分镜像层与某节点上已缓存的其他镜像的部分镜像层相同。

在情形一中，本文以基础镜像为 debian 的 nginx 为测试镜像部署容器，其中 debian 镜像大小为 80.4MB，nginx 镜像大小为 131MB。任务通过单个 Pod 的方式部署，各 worker 节点缓存、待调度任务及调度结果如下表 3.5 所示。

表 3.5 节点缓存、待调度任务及调度结果

节点名/Pod 名	缓存镜像/所需镜像	节点得分/调度结果
worker1	debian:latest	55
worker2	无	0
worker3	无	0
test2	nginx:latest	worker1

实验结果表明，拥有基础镜像的节点获得了更高的评分，本方案可以准确将任务调度到该节点。而默认调度器则不会考虑基础镜像的复用，由于本实验环境各节点配置相同，默认调度器调度实验结果为随机选取。在时延方面，本方案由于避免了基础镜像的重复下载，从而优于默认调度器。

在情形二中，本文以 redis:6 与 redis:7 为测试镜像部署容器，其中 redis:6 镜像大小为 113MB，redis:7 镜像大小为 117MB。各 worker 节点缓存、待调度任务所需的镜像及调度结果如下表 3.6 所示。

表 3.6 节点缓存、待调度任务及调度结果

节点名/Pod 名	缓存镜像/所需镜像	节点得分/调度结果
worker1	无	0
worker2	redis:6	77
worker3	无	0
test3	redis:7	worker2

实验结果表明，redis 镜像不同版本间有着 77%的缓存重复度。本方案可以充分利用不同版本镜像之间相同的镜像层缓存，将任务调度到拥有低版本镜像的节点。默认调度器依然为随机选取。在时延方面，本方案仅下载了镜像不同版本间所差异的镜像层，从而优于默认调度器。

在情形三中，本文以 nginx 与 redis 镜像为测试镜像部署容器，二者都来源于相同的基础镜像，但该基础镜像本身不直接存在于任一节点。各 worker 节点缓存、待调度任务及调度结果如下表 3.7 所示。

表 3.7 节点缓存、待调度任务及调度结果

节点名/Pod 名	缓存镜像/所需镜像	节点得分/调度结果
worker1	无	0
worker2	无	0
worker3	redis:7	31
test4	nginx:latest	worker3

实验结果表明，nginx 与 redis 镜像有着 31%的镜像层重复。本方案在调度时能够准确识

别不同镜像间可复用镜像层，并将任务调度到对应节点上。而默认调度器则无法识别，结果为随机选取调度节点。在时延方面，本方案一定程度上减少了镜像下载量，从而优于默认调度器。

3.4.4 镜像层缓存完全命中场景

该场景下部分节点已经拥有完整的所需镜像。在调度单个 Pod 且各节点负载状况相同时，本方案的部署速度与 Kubernetes 默认调度策略并无显著差异，但在调度多个 Pod 时具有较好的性能提升。本实验创建名为 test5 的 Deployment 调度多个 Pod 并设置副本数为 3，调度 Pod 名称记为 testA~C，各 Pod 所需镜像均设为 nginx。各 worker 节点缓存、待调度任务及调度结果如下表 3.8、表 3.9 所示。

表 3.8 节点缓存及得分

节点名	缓存镜像	节点得分
worker1	无	0
worker2	nginx:latest	100
worker3	无	0

表 3.9 待调度任务及调度结果

Pod 名	所需镜像	本方案调度结果	默认调度器调度结果
testA	nginx:latest	worker2	worker2
testB	nginx:latest	worker2	worker1
testC	nginx:latest	worker2	worker3

实验结果表明，在某节点拥有待调度任务所需的完整镜像时，本方案会首要考虑镜像层缓存的复用，并始终将任务调度到拥有完整镜像的节点。而默认调度器尽管包括了 ImageLocalityPriority 策略，会在一定程度上让已经拥有所需镜像的节点获得更高评分，但是其更多考虑了 CPU、内存、负载均衡等因素，在调度多个 Pod 时依然更倾向于将任务分散到各个节点。在时延方面，本方案通过复用节点上的完整镜像，避免了镜像下载阶段带来的时延，而默认调度器仅有一次利用了节点的缓存，因此在部署速度上优于默认调度器。

3.4.5 时延对比与总结

经测试，本文调度器在上述实验中的调度时延约为 1.3s，主要用于从镜像仓库下载镜像的 manifest 文件。但下载 manifest 文件的过程是一次性的，通过存储镜像名与镜像层的对应关系信息，调度器在后续调度需求相同镜像的 Pod 时可以大幅减少调度时延。在已缓存对应

关系信息的前提下，相比于对镜像层进行逐一匹配并实时与 API 交互的调度方式，本方案所减少的调度时延百分比如表 3.10 所示，其中 test1~test5 分别对应了上述不同缓存命中场景下的 5 组实验。其主要源于本方案在某一所需镜像层未命中节点缓存时，跳过了该节点剩余层的对比过程，并且在调度前预先将节点缓存信息同步到调度器本地，减少了调度器与 API 实时交互的时延。

表 3.10 不同场景下本方案所减少的调度时延百分比

test1	test2	test3	test4	test5
20%	17%	16%	17%	19%

在容器整体部署时延方面，图 3.10 展示了在上述实验中使用本文调度器与默认调度器 Kube-Scheduler 的容器整体部署时延对比。为避免网络波动影响，结果为多次测试后的平均时延。

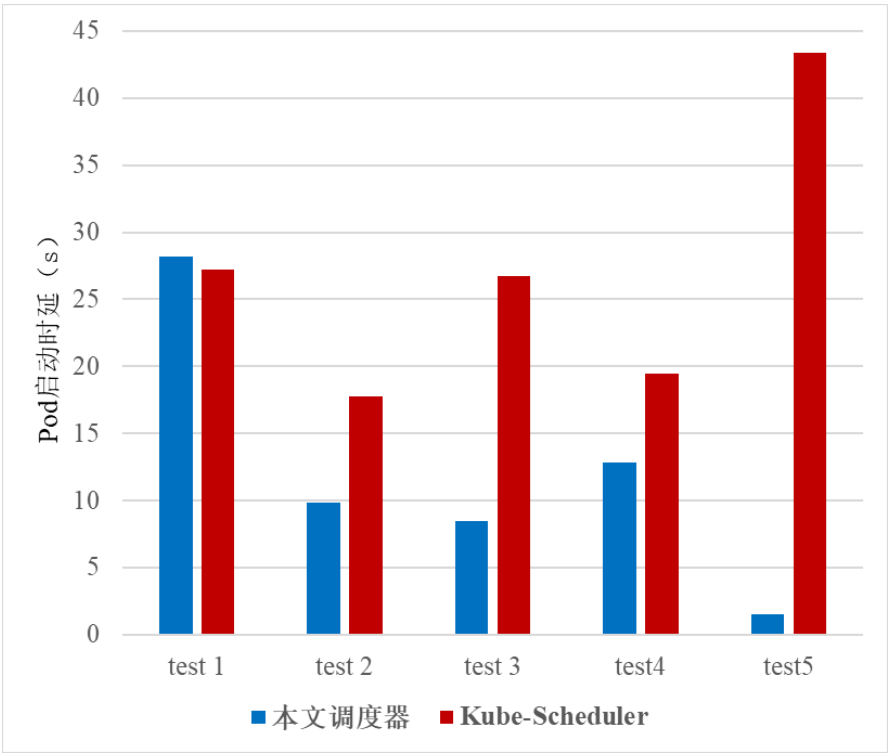


图 3.10 不同场景下容器整体部署时延对比

结果表明，本文调度器在容器冷启动的情况下，实质上等同于随机调度，同时未考虑默认调度器所关注的各项节点指标，性能不及默认调度器。但在节点拥有所需镜像层缓存的情况下，本文调度器可以高效利用节点缓存，减少下载量，因此在容器部署时延上有所提升，提升幅度取决于任务所需镜像和节点缓存镜像层的命中量。同时，在面对启动多个相同任务的 Pod 任务场景时，本文调度器仅在第一次需要下载节点所缺乏的镜像层，而后续则可以完整复用该节点的镜像，因此具有较大的性能提升。综上所述，本文方案可以有效减少镜像下载量从而提升容器部署速度。

3.5 本章小结

本章设计了一种基于节点镜像层缓存的容器调度方案。首先收集节点缓存的镜像层信息并同步给调度器，随后调度器将任务所需镜像层与节点缓存进行对比，进而将任务调度到镜像层缓存命中率最高的节点上。本文调度框架基于 Kubernetes 架构，通过自定义 CRD 与 Controller 收集镜像层信息，并基于 Kubernetes 调度器拓展实现了相应的调度功能。实验结果表明，本文方案具有良好的调度效率，并且在节点拥有所需镜像层缓存时，能有效减少镜像下载量，从而提高容器部署速度。尤其在缓存命中率较高以及调度多个相同 Pod 的情况下，具有较好的性能提升。

第四章 基于 IPFS 的分批式镜像分发方案

本章的研究聚焦于通过 P2P 传输方式以加速镜像分发。采用了基于内容寻址的 IPFS 协议以实现去中心化的镜像传输方案。在此基础上，依据集群节点的镜像获取情况，动态地将节点加入 IPFS 网络进行镜像传输，加快了节点平均容器启动速度并提升了传输过程的鲁棒性。

4.1 背景分析

在容器集群中，随着服务规模的扩大，任务需要被调度到多个不同的节点上以实现负载均衡和高效的资源利用，因此多个节点需要下载相同的容器镜像。在对服务响应速度有严格要求的任务场景中^[71]（如电商平台进行促销活动、流媒体服务发布新热点内容等），镜像分发的速度成为了影响服务质量的关键。此类任务场景需要减少用户等待时间的同时保障服务规模逐步扩大，在初始阶段，集群为快速响应突然增加的用户访问量，需要少量节点迅速启动容器以应对早期流量，避免用户等待过久，而在后续阶段，为了满足不断增长的流量需求，必须确保剩余多数节点也能以较快速度启动。

默认的镜像分发方式如图 4.1 所示，在面对上述场景时，多个节点会并行地从中心镜像仓库下载镜像，可能会导致镜像仓库的网络资源拥堵，影响每个节点的下载速度和服务启动时延，因此该方案在这种高并发和需要快速扩展的场景中尤为不利。为了解决中心仓库的网络拥堵问题，业界普遍借助 P2P 传输协议（如 BitTorrent 和 IPFS）进行镜像分发，使得镜像数据能够在节点间直接传输，减少对中心镜像仓库的依赖，提升总体下载速度。

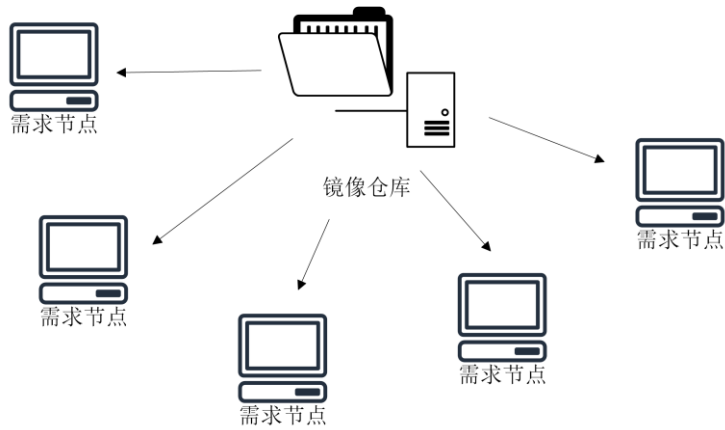


图 4.1 基于镜像仓库的镜像分发网络模型

4.1.1 基于 BitTorrent 的镜像分发

在使用 BitTorrent 进行镜像分发时^{[34][36]}, 需要 Tracker 记录、管理 P2P 网络中各节点的相关信息。当节点需要某镜像时, 需要先下载镜像对应的种子文件, 随后与 Tracker 交互并得知存有所需镜像文件的节点, 最后与上述节点直接建立连接下载所需镜像文件。

此方法减少了多节点下载请求时的带宽竞争, 在效率上相比于从镜像仓库下载提升许多, 但同时也有其弊端:

(1) 依赖 Tracker 服务器。Tracker 作为中心服务器, 需要解析种子文件, 为需求节点提供拥有所需资源节点的信息。虽然 Tracker 提供拥有镜像的节点信息所产生的负担相较于直接从中心下载镜像已减少很多, 但在并发请求量较高时, 依然会积累较长的处理时延。同时若 Tracker 因故障等原因无法提供服务, 则会影响集群整体的镜像分发流程。

(2) 镜像种子文件带来了额外的开销。在实际应用中, 镜像的种类与数目繁多, 导致种子文件的构建与管理变得复杂。并且在传输前, 节点需要先下载对应的种子文件。该文件虽然体积较小, 但是在下载时建立连接、释放连接的过程不可避免, 也会带来一定时延。此外, 在面对小体积镜像下载请求时, 种子文件的体积占有不可忽视的比例, 传输种子文件的开销也相对上升。

综上所述, 基于 BitTorrent 协议的镜像传输方案虽然在很大程度上提升了镜像分发的整体速度, 但是在其稳定性及传输开销等方面依然会遇到瓶颈。

4.1.2 基于 IPFS 的镜像分发

针对 BitTorrent 的不足, 部分研究者采用 IPFS 作为 P2P 镜像分发工具以实现完全去中心化的同时无需种子文件^[72-73], 从而更好地适应多节点并发拉取镜像的场景。

本文总结了基于 IPFS 的镜像分发总体流程, 如图 4.2 所示。首先, 在集群中各节点配置 IPFS 组件, 将其加入到同一个私有 IPFS 网络中, 随后启动 IPFS daemon 提供 P2P 网络服务。其次, 当集群中有节点需要某镜像时, 由拥有该镜像的节点导出并共享镜像文件, 如果所有节点都未拥有所需镜像, 则随机挑选一个节点从镜像仓库下载。在共享后, IPFS 会生成一个该镜像文件对应的 hash 值。接着, 需求节点根据此镜像文件的 hash 值从 IPFS 网络中下载镜像到本地。最后, 节点的容器运行时将镜像文件导入本地镜像存储系统中。

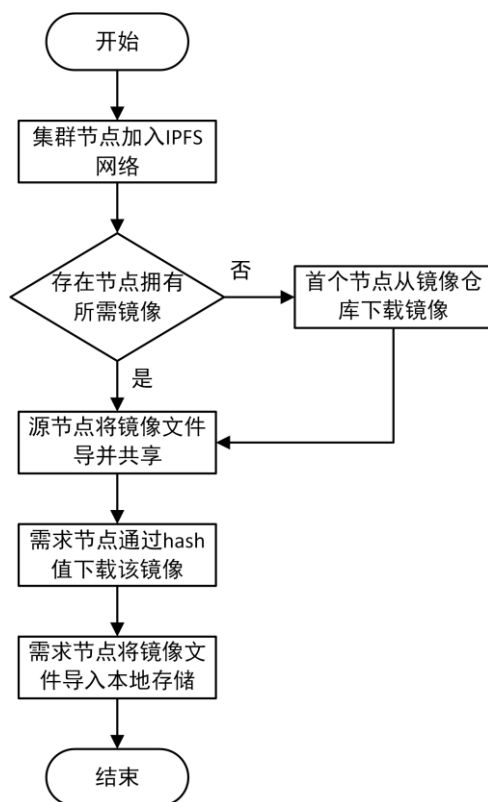


图 4.2 基于 IPFS 的镜像分发

上述方案有效解决了 BitTorrent 依赖 Tracker、种子文件带来额外开销的问题，一定程度上提升了稳定性，减少了传输开销。但在所述场景中，该方案仅提升了镜像的整体分发速度，无法满足部分节点的快速响应需求，同时在传输效率和鲁棒性方面仍存在不足：

首先，虽然 P2P 传输方式在整体上提高了镜像分发速度，但是所有节点几乎都在同一时刻完成镜像下载、启动容器服务，从而在最初的一段时间内，服务始终处于不可用的状态，增加了用户等待时间，一定程度上影响了服务质量，因此上述方案在面对需要部分节点快速启动的任务场景时表现不佳。

其次，IPFS 在传输镜像时，需要在网络中通过 DHT 逐级查找拥有所需镜像的节点，此节点发现的过程需要一定时间。尤其在源节点数量相对较少的情形，集群中大多数的节点仅作为路由转发的中间节点，此类节点占据了 IPFS 网络的主要部分，其不仅无法提供所需文件，还使得网络拓扑变的更加庞大与复杂，从而导致节点发现的过程更为缓慢。

最后，在最初下载阶段，需求节点只能从有限的下载源选择中获取镜像文件块，可能会导致下载者之间竞争源节点的网络资源，传输质量容易受到初始节点网络波动的影响。此外，虽然 IPFS 使节点在下载完所需文件块后即时共享，从而后续的下请求拥有更多的下载源。但是镜像块的下载顺序是随机的，因此难以保证所需的文件块存在于多个节点上。当初始源节点退出集群时，其他节点可能无法获取到剩余所需文件块，从而导致传输过程发生阻塞。

4.2 基于 IPFS 的分批式镜像分发策略设计

本章的镜像分发方案目标应用于部分节点需要更快响应的任务场景，以提高镜像分发效率为核心，同时提升镜像传输时的鲁棒性，从而加速容器部署，提升服务质量。本方案在设计上主要考虑以下因素：

（1）避免中心式的网络结构。由于高并发请求往往给中心网络节点带来较高的负担，而中心节点的性能往往会影响着集群整体效率。为保障可靠性和高效性，应将网络负载均衡到集群中的各个节点上。

（2）减少管理和传输的开销。面对种类繁多的任务请求，需要设计高效的方案统筹与管理镜像分发。

（3）提高镜像分发和服务响应速度。在并发请求较高时，相比于现有镜像分发方案，本方案需要使部分节点更早提供服务，同时在镜像平均下载时延上有所提升。

（4）提升分发过程的鲁棒性。由于在镜像下载时可能有部分节点出现网络波动或节点故障的情况，在设计时需要减少因少量节点的波动或故障对其他正常节点的影响。

针对上述需求，本文镜像分发方案以 IPFS 协议作为底层传输协议，并在此基础上设计了一种分批式镜像分发策略：

在开始传输前，考虑到集群中存在不同类型的节点，包括较少的源节点，较多的需求节点，以及不参与任务的其他节点，本方案首先构建拥有镜像以及需求镜像的节点列表，依据此列表避免在后续将其他不参与任务的节点加入 IPFS 网络。在传输时，本方案并非直接将所有源节点和需求节点一次性加入 IPFS 网络。而是先由源节点构建最初的 IPFS 网络，接着依据源节点的数量，加入等量的需求节点进行镜像下载。待这些需求节点完成下载后，随即启动容器服务，并共享该镜像以作为新的源节点参与到后续分发流程中。随后依据更新后的源节点数量，再次加入等量的需求节点进行下载。重复上述操作，直到集群中所有需求节点都获取到该镜像。该策略的执行流程如图 4.3 所示。

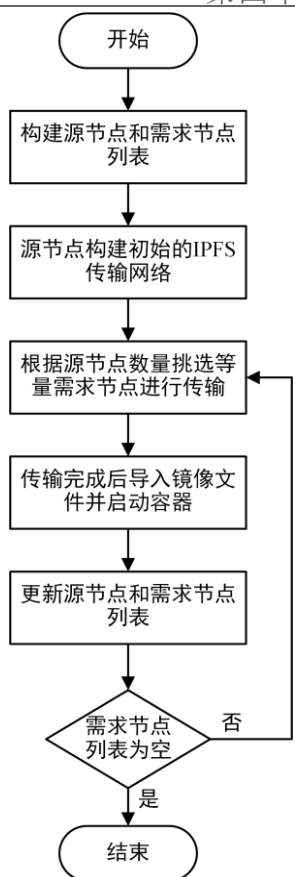


图 4.3 基于 IPFS 的分批式镜像分发策略

如图 4.4 所示，在 P2P 传输的初始阶段，虽然集群中源节点数量较少，但是本方案只加入了与其数量相等的需求节点，因此网络结构较为简单，节点发现与路由选择较为迅速。而在 P2P 传输的后期，虽然此时网络集群的规模较为庞大，但是源节点的数量也占有相当高的比例，使得新加入的节点通过较少查询次数便可发现源节点，同时可以在众多源节点中，选择距离较短、网络状况较好的传输路径，从而提升传输效率。而将节点统一加入 IPFS 网络时，在极端情况下，可能需要搜寻整个集群网络才能找到拥有所需镜像的节点，从而导致较长传输时延。

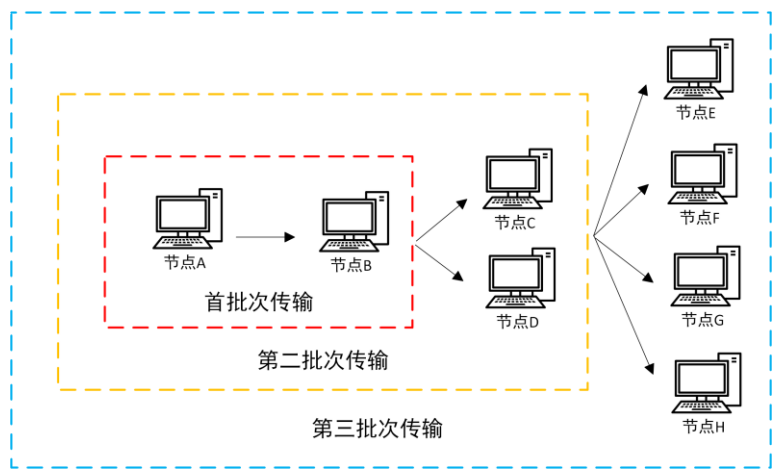


图 4.4 分批式镜像分发示意图

与此同时，由于在新加入节点前，先前的节点已经获取到完整的镜像文件，这些节点可以更早启动容器，从而加快了服务的响应速度。并且这些节点成为了新的下载源，保障了后续需求节点可以稳定地从多个源节点处下载所需的镜像文件块，减少了对同一源节点网络的带宽资源竞争，进而加快了镜像传输速度。此外，在传输过程中，越来越多的多个节点拥有完整的镜像副本而并非只是部分文件块，即便部分节点因网络故障等原因而无法提供下载，剩余节点也可以从拥有该副本的其他节点处下载镜像文件，有效增加了传输过程的鲁棒性。

4.3 基于 IPFS 的分批式镜像分发框架设计

在实际应用层面，通过手动的方式进行上述方案的部署与配置会带来较大的开销。首先在进行网络配置时，每个节点都需要查询其他节点 IP 信息并在各自节点上执行信息添加操作，从而将自身加入到 IPFS 网络中。当某节点因故障等原因不可用或退出集群时，需要在其他节点上删除该节点对应的信息。此外，当指定某镜像进行分发时，需要先在集群中找寻已经拥有该镜像的节点并使其上传镜像文件。在传输过程中，还需时刻关注节点镜像传输完成情况，及时将后续的需求节点加入到 IPFS 网络中。上述的流程会造成较高的运维难度和成本。

为减少部署与配置的开销，本文依据 4.2 节所述方案，设计了一种基于 IPFS 的镜像分批传输框架，该框架如图 4.5 所示，该框架可以快速地获取各节点的 IP 信息和镜像缓存情况，在传输前通过远程访问的方式使拥有所需镜像的节点配置网络、启动传输服务并共享镜像，并在传输过程中实时监测集群中各节点的镜像下载情况，在同批次节点完成下载后快速协调后续节点的下载。

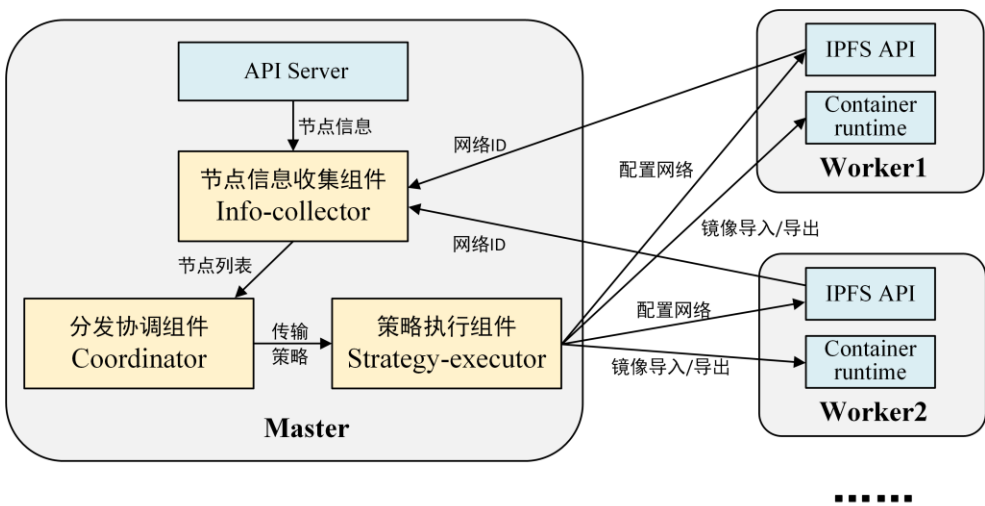


图 4.5 基于 IPFS 的分批式镜像分发框架

其主要包括以下三个组件：

- (1) 节点信息收集组件（Info-collector）。该组件与 Kubernetes API Server 交互，查询集

群各节点的 IP 地址, 根据节点 IP, 与各节点上的 IPFS 组件进行交互并返回节点的网络 ID 信息。同时, 根据指定的镜像名及版本号, 判断该节点上是否已存在所需的镜像, 进而构建最初的源节点列表和需求节点列表。

(2) 分发协调组件 (Coordinator)。该组件用于实时监测源节点的数量变化以及需求节点的镜像下载状况, 根据算法逻辑规划节点的镜像分发顺序, 分批次地将需求节点加入到最初的 IPFS 网络中。由于不同节点的网络状况各不相同, 下载镜像的时延也不相同, 因此该模块统筹了同一批次的节点下载进度。该组件会循环运行直到需求节点列表为空。

(3) 策略执行组件 (Strategy-executor)。该组件根据收集到的节点 IP 和网络 ID, 通过远程访问的方式与初始源节点的容器运行时交互, 使其导出所需的镜像文件。随后与 IPFS 组件进行交互构建网络集群, 并将镜像文件上传至 IPFS 网络中。当收到新节点的传输指令时, 该组件将相关节点信息添加到新节点配置中并启动 IPFS 服务进程。此外, 当需求节点完成所需镜像的下载后, 该模块将通知节点的容器运行时将该镜像文件导入到本地镜像存储中。

框架整体工作流程如图 4.6 所示。

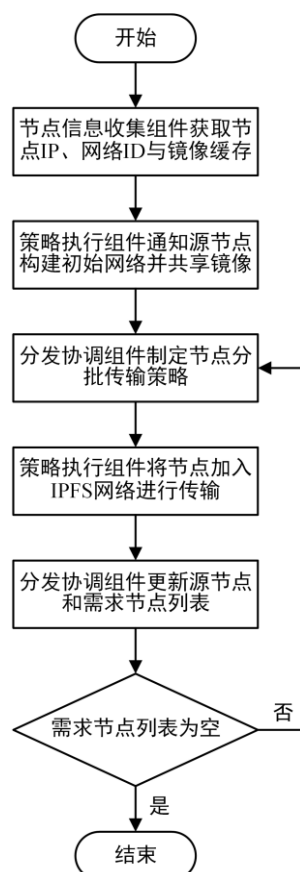


图 4.6 分批式镜像分发框架工作流程

首先, 针对某一特定镜像, 节点信息收集组件与 Kubernetes API Server 交互, 查找拥有该镜像的节点并构建源节点列表。倘若集群中所有节点都尚未拥有此完整镜像, 则先选取一个节点通过镜像仓库获取完整镜像。其次, 策略执行组件通过远程访问的方式与上述节点的容

器运行时及 IPFS 组件交互，构建初始的 IPFS 网络并将镜像文件导出、共享。紧接着，分发协调组件根据集群中当前源节点数量，通知策略执行组件将等量的需求节点加入到 IPFS 网络中进行镜像传输。在传输完成后，策略执行组件通知节点将镜像文件导入并启动容器，同时为后续需求节点提供下载服务。最后，分发协调组件更新源节点列表和需求节点列表，并重复上述的操作直到所有需求节点完成镜像获取。

4.4 实验与分析

本节通过数个实验以评估该镜像分发方案的可行性和高效性。首先介绍了本方案的部署平台及配置状况，随后针对不同分发场景进行测试。在对比阶段，本文首先针对不同的节点规模，在总体镜像传输时延上与中心镜像仓库和改进前的 IPFS 镜像分发方式进行对比，接着为验证本策略在面向需要快速响应的任务类型时的优势，记录了传输过程中各时间段的节点传输完成情况，计算了节点平均传输时延并与改进前的方案进行对比。

4.4.1 实验环境

本文通过虚拟机的形式在物理机上共部署了 10 个节点，其中节点的配置如表 4.1 所示。其中 master 为主控节点，worker 为工作节点，registry 作为中心镜像仓库，存储了实验中所需下载的镜像。

表 4.1 实验环境配置

节点名称	系统	CPU 核数	内存 (GB)	带宽 (Mbps)
master	Centos7	4	6	1000
worker1~worker8	Centos7	2	4	1000
registry	Centos7	2	4	1000

上述节点均已安装并配置了 Kubernetes 容器编排引擎并处于同一集群中，其中 Kubernetes 版本为 1.19，底层容器运行时为 Docker 19.03，此外在集群中各节点均已安装 go-ipfs v0.26.0 的相关组件。框架各模块均以插件化的形式从 master 节点部署到集群中。

此外，由于在真实的网络情景中，互联网服务提供商往往采用不对称的网络连接方式，设计上假设节点会更多地下载内容（如网页浏览、视频流、文件获取等），而较少地上传内容。因此，集群节点往往拥有较高的下载速度和较低的上传速度。本实验为模拟真实场景的网络情况，通过 tc 指令将节点上传带宽限制为 200Mbps，而下载带宽则不做限制。

4.4.2 结果对比与总结

首先, 本文分别测试了在不同数量节点发起并发下载请求时, 通过中心镜像仓库、传统 IPFS 和本文改进方案进行镜像分发的传输时延。实验中分发的镜像大小为 895MB, 在使用传统 IPFS 和本方案下载时, 已事先于 worker1 节点下载所需镜像并作为初始源节点, 并在不同数量的其他 worker 节点上进行分发。实验结果如图 4.7 所示。

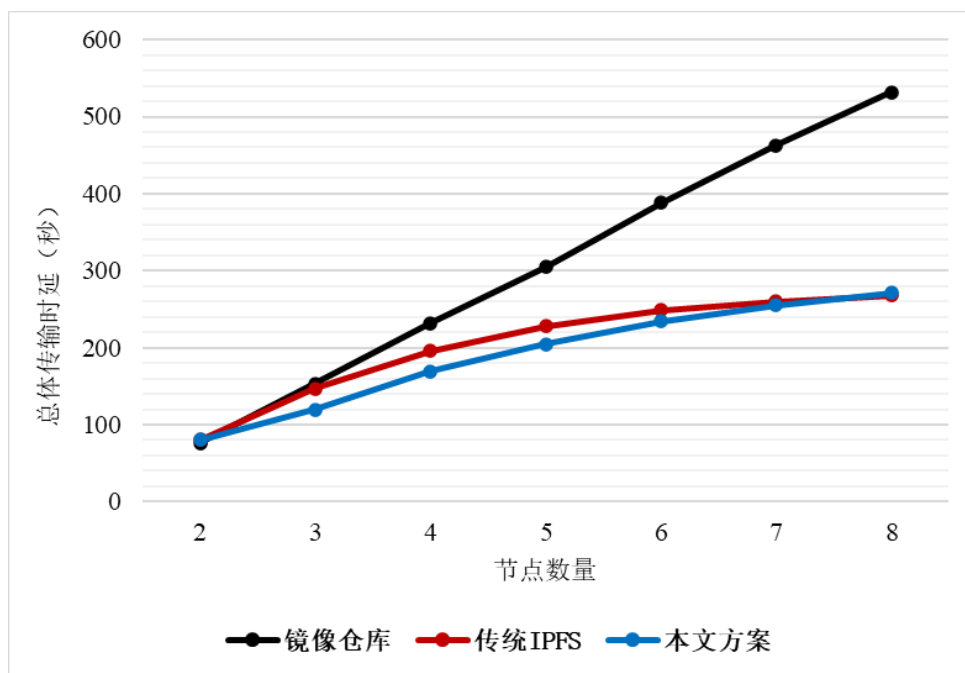


图 4.7 镜像总体传输时延对比

结果表明, 当只有单个节点下载镜像时, IPFS 镜像分发速度不及从中心镜像仓库下载, 原因在于 IPFS 节点间路由发现与路径选择会带来额外的时延。但随着多节点开始并发请求下载, IPFS 发挥了其利用节点间带宽的优势, 在大于等于 3 节点的情况下, 镜像传输速度超越了中心式下载, 提升幅度随着节点规模的增加而增加。在对比传统 IPFS 与本文改进方案时, 在节点数量较少时, 本文方案保障了后加入的节点可以稳定从多个不同的源节点下载不同的文件块, 从而进一步提升了总体传输速度, 但在节点数量规模较大时, 由于后续批次需要等待前一轮传输完成, 因此带来了一定的等待时延, 性能有所下降。

其次, 为验证本文方案在节点响应速度方面的优势, 实验将本方案与传统 IPFS 传输方式进行对比, 首先分别记录了 8 节点数量下二者在不同时间段内的传输完成情况。随后对比了二者在不同节点数量下的平均传输完成时间。实验结果如图 4.8、图 4.9 所示。

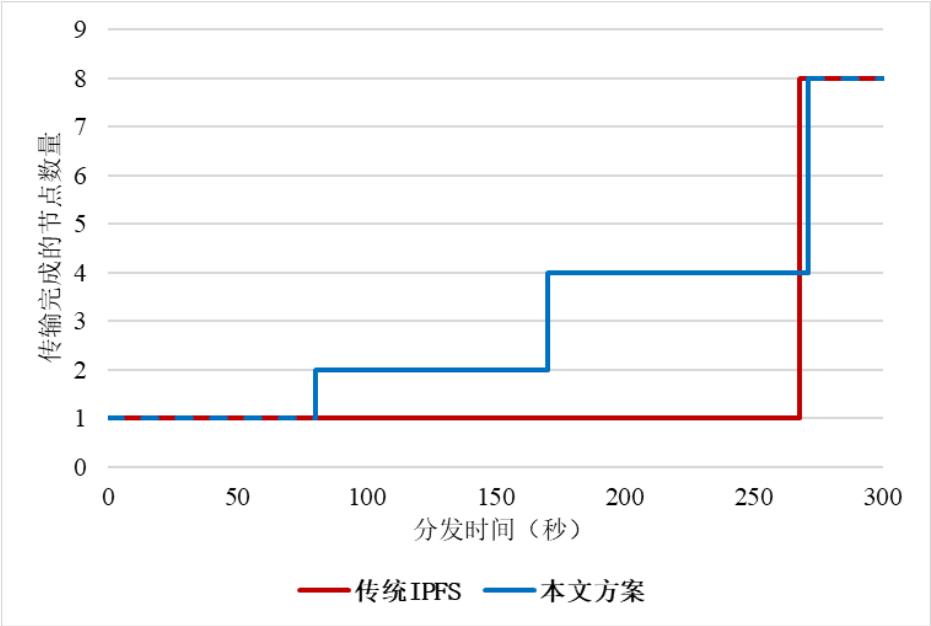


图 4.8 不同时间段内传输完成的节点数量

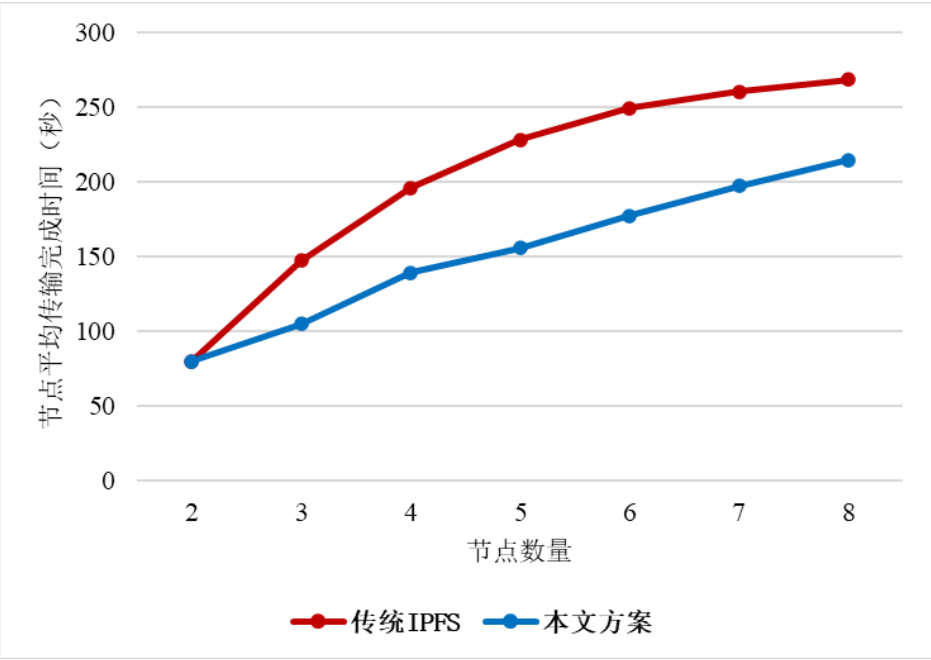


图 4.9 节点平均传输完成时间

结果表明，相比于使用传统 IPFS 进行镜像分发，本方案有 50% 的节点能够更早完成传输，从而更快地启动容器、响应任务需求。此外，即使是在总体时延传输时延劣势的情况下，本文方案在节点平均传输速度上依然有较大提升。其主要源于在传输过程中，本方案将有限的源节点带宽优先集中在部分节点上并使其更快获取完整的镜像，并且在加入新节点前，已经有多个节点获取了完整镜像，保障了新加入的节点可以并行地从多个节点下载不同的文件块，从而加速了这些节点的传输速度，这些更快完成传输的节点显著降低了集群节点的平均传输时延。

综上所述,本方案能够使部分节点更快地响应任务需求,加快了平均容器启动速度。因此可以较好适用于部分节点需要更快响应的任务场景。此外,在节点数量较少时,本方案相比于直接使用 IPFS 进行镜像传输在总体时延上有所提升。与此同时,在本方案的传输过程中,有更多节点拥有完整的镜像副本,从而在初始源节点故障时依然能够保障其他节点的稳定传输,有效提升了分发过程的鲁棒性。

4.5 本章小结

本文面向对服务响应速度有严格要求的任务场景,提出了一种基于 IPFS 的分批式镜像分发方案。通过在节点之间传输镜像,缓解了中心镜像仓库的压力。在此基础上,通过分批传输的方式优先保障了部分节点容器的快速启动。实验结果表明,相较于传统 IPFS 分发方式,本方案减少了部分节点的响应时延,提高了容器的启动平均速度并提升了传输过程的鲁棒性。此外,在节点规模较小时,本方案有效减少了镜像分发的总体时延。

第五章 基于 Kubernetes 的容器部署多阶段加速系统

本章面向云原生环境下容器部署场景，将本文第三、四章的相关技术路线进行融合，在任务到来前以及容器部署中的调度和镜像分发阶段设计了高效方案，最终基于 Kubernetes 实现了一个完整的容器部署多阶段加速系统。

5.1 需求分析

在现今的技术领域，容器化已经成为了应用部署的主流选择。然而，随着任务需求变得更加复杂，容器部署面临的挑战也在增加，例如不同任务对资源的需求各不相同，其服务规模、所需镜像大小及节点网络状况也会影响镜像获取速度。因此，在面对复杂的任务场景时，保障容器的部署效率、灵活性以及稳定性成为了企业提升服务质量的关键。

虽然 Kubernetes 默认的容器部署方式能够处理基本的应用需求，但在资源调度、镜像分发以及服务启动等方面仍存在明显的不足。首先，在资源调度方面，Kubernetes 采用的调度策略较为单一，在面向具有复杂需求的任务场景时，存在资源利用率低和响应时间长等问题。其次，关于镜像分发，由于缺乏有效的预测机制，新启动的服务需要先等待镜像下载，严重影响了部署速度和用户体验。与此同时，现有的镜像分发流程未能根据任务需求及集群实时状态进行灵活调整，带来了额外的分发时延，导致容器部署速度缓慢。最后在服务启动上，现有方案主要关注集群整体的部署效率，难以适用于部分节点亟需更快启动的任务场景。

在此背景下，构建一个高效、灵活的容器部署加速系统变得至关重要。本章探讨了在多种任务场景下容器部署加速的相关策略及其具体实现。重点论述在任务到来前的准备阶段、容器调度阶段、以及镜像分发阶段的优化策略，并将这些阶段以自动化、流程化的方式结合，最终构建一套完善的容器部署加速系统。该系统旨在依据任务需求和集群实时环境，选择最优的容器部署方式，并以系统化的处理流程简化部署操作，从而加速服务更新与迭代，使得业务能够迅速适应市场变化。

5.1.1 系统功能性需求

基于上述分析，本文的容器部署加速系统主要需要以下功能：

（1）镜像缓存功能

在任务到来前，系统根据集群现有容器运行和镜像缓存情况，判断集群当前的热门镜像，同时结合用户指定的镜像，构建镜像预拉取列表。并通过设置相应的网络监控组件，检测集

群的实时网络负载情形，选择合适的时间段进行镜像预拉取。

（2）容器调度功能

采用灵活、高效的调度算法，根据不同任务的需求，在满足任务对资源利用率、节点负载均衡等要求的前提下，充分利用集群节点已有的镜像层缓存，最终权衡多种指标选择最适合的工作节点。

（3）镜像分发功能

在调度完成后，根据容器所需的镜像大小、运行容器的节点规模以及中心仓库的网络负载状况，在中心式和 P2P 下载方式中做出自适应决策。当采用 P2P 镜像传输时，根据节点数量、任务对总体部署时延和部分节点快速响应的要求，选择合适的节点分发顺序。同时依据节点网络和镜像层缓存情况，选择最优的初始节点并避免镜像层的重复下载。

（4）日志收集功能

在一个完整的容器部署流程结束后，输出对应日志，其中包括任务的需求信息、系统在部署该任务时所做的各项决策、各模块所设的指标权重以及最终调度结果和时延，便于用户了解容器部署过程的详细信息以对系统参数、指标等进行后续调整。

5.1.2 系统非功能性需求

系统除了具备上述的功能外，还需满足以下非功能性需求：

（1）灵活性

在设计系统架构时，应简化系统部署流程以适应不同集群的环境。同时系统在面对各式各样的容器部署场景时，可以灵活调整部署策略，满足不同的任务需求。

（2）稳定性

在设计系统方案时，应采用高容错机制，在部分节点发生故障时，保障其他节点的部署流程不被中断。同时需要完善的监控和报警功能，使运维人员能够实时了解系统状态并迅速处理问题。

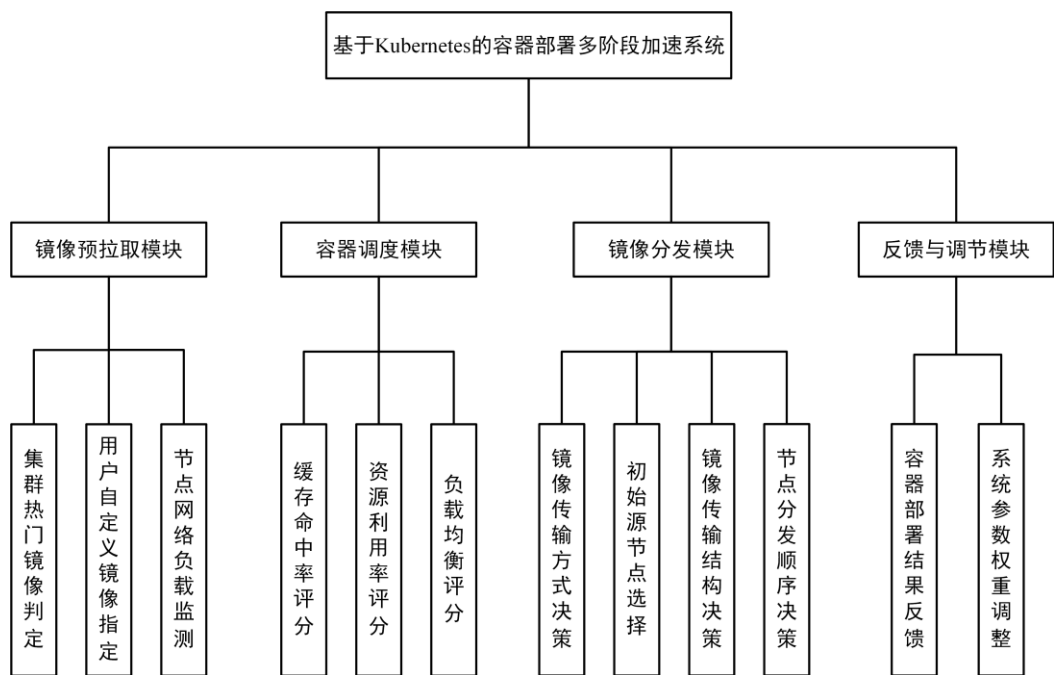
（3）可拓展性

系统应采取不入侵原有容器集群代码的原则，通过插件的方式实现容器部署过程的灵活选择，保障集群原有功能的同时满足不同的部署需求。此外系统应采取模块化设计，使得新的部署策略可以轻松集成进现有架构中以便于单独升级和维护，同时为将来的功能扩展提供便利。

5.2 系统设计与实现

本文前两章详细探讨了在容器调度阶段和镜像分发阶段的相关策略，为解决容器部署的效率问题提供了切实可行的解决方案。在实际应用中，任务时往往有着各种各样的部署需求，为满足不同任务场景，本节对前两章策略进行高效融合，在此基础上设计了相应组件并完善了部署策略，最终设计了一个基于 Kubernetes 的容器部署多阶段加速系统。

本文系统基于 Kubernetes 平台，主要分为镜像预拉取模块、容器调度模块、镜像分发模块和反馈与调节模块。在配置该系统之前，需要在集群中预先部署 Prometheus 组件^[74]以收集节点的实时网络负载信息。系统整体概览如图 5.1 所示。



其中，镜像预拉取模块在部署任务到来前预先构建节点镜像缓存；容器调度模块在第三章工作的基础上，通过一种多指标加权融合的调度方案，高效地选择任务执行节点；镜像分发模块在第四章工作的基础上，通过一种自适应调节的镜像分发方案，快速地分发任务所需镜像；反馈与调节模块反馈系统运行时的数据和指标，方便用户对系统进行调优。本节后续将具体介绍系统各模块的原理及工作流程。

5.2.1 镜像预拉取模块

镜像预拉取模块用于在部署任务到来前提升集群节点的缓存命中率。在判断节点网络负载较低时，该模块对一些未来可能使用到的镜像进行预下载，从而在不影响现有任务的同时增加后续任务的缓存命中率。在集中的任务部署请求到来时，系统可以有效利用这部分缓存，

南京邮电大学硕士研究生学位论文 第五章 基于 Kubernetes 的容器部署多阶段加速系统

从而减少镜像分发阶段的下载量,提高容器部署速度。此模块通过 Kubernetes 自定义 Controller 实现,框架如图 5.2 所示。

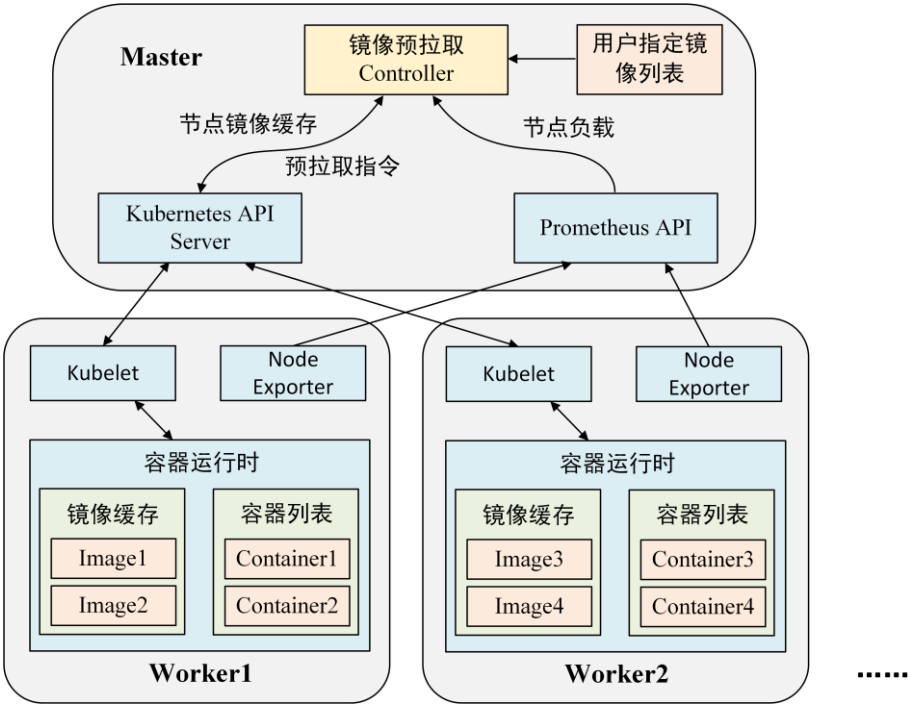


图 5.2 镜像预拉取模块架构图

该模块工作流程如图 5.3 所示，首先，控制器会与 Kubernetes API Server 交互，获取反馈节点缓存的镜像信息，进而统计集群中每个镜像的缓存频率，其中已缓存于较多节点的镜像会被标注为热门镜像并作为预拉取的目标。此外，考虑到任务实时性和弹性扩容需求，控制器会收集集群中正在运行的容器信息，对于运行中容器所需的镜像，会额外统计一次缓存频率。同时，该模块也支持用户对预拉取镜像的自定义指定，例如一些常用的 `alpine`、`centos` 等基础镜像，或者是即将上线的业务所需的镜像。随后，该模块会将未拥有上述镜像的节点加入统一列表并在后续拉取镜像。为避免镜像预拉取占用过多网络资源，导致对现有服务造成影响，本模块通过 Prometheus API 与各节点的 Node Exporter 交互，使用 PromQL 查询节点在一定时间段内下载的字节数，从而判断节点当前的网络负载状况。当列表中的节点在一段时间内的网络负载持续低于一定阈值时，则会向该节点发出镜像预拉取指令。在拉取镜像时，该模块会依据镜像名和节点名，创建对应的 job 类型任务，任务的 yaml 文件示例如图 5.4 所示。该任务会在指定节点上下载镜像并在下载完成后输出提示信息。

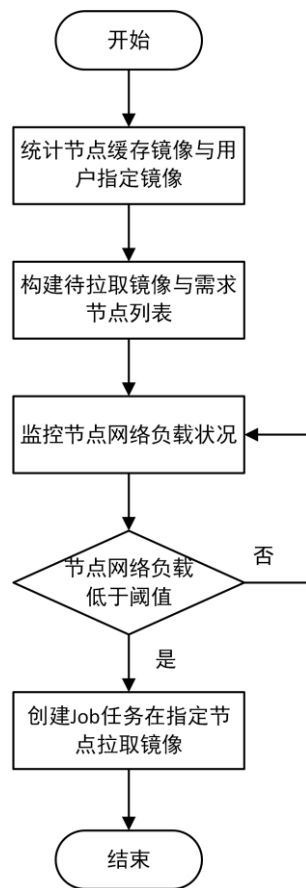


图 5.3 镜像预拉取模块工作流程

```
apiVersion: batch/v1
kind: Job
metadata:
  name: image-prepull
spec:
  template:
    spec:
      containers:
        - name: prepull-container
          image: nginx:latest
          command: ["sh", "-c", "echo 'Image pulled successfully'"]
          nodeName: worker1
          restartPolicy: Never
      backoffLimit: 1
```

图 5.4 镜像预拉取任务 yaml 文件示例

5.2.2 容器调度模块

容器调度模块旨在根据任务需求将 Pod 调度到合适的节点，同时有效减少镜像的下载量，提高容器的部署速度。在第三章基于节点镜像层缓存的容器调度策略中，本文通过记录节点镜像层缓存信息，在调度时将任务所需的镜像层与节点的镜像层缓存信息进行对比，将任务

调度到拥有镜像层缓存最高的节点上。该方法在面对规模较小的任务类型时具有显著的性能提升。但考虑到在任务规模较大时，大量的任务会被调度到单个节点，容易导致任务资源需求超出节点的承载能力或集群负载不均。为提升系统的灵活性，本模块提供了一种具有可调节性的调度方案以满足不同类型的任务需求。

本文通过 scheduler framework 设计了自定义 filter 和 score 策略，实现了一种在不超出节点资源承载能力的前提下，根据任务需求权衡节点镜像层缓存命中率、资源利用率以及负载均衡指标的调度器。当收到待调度任务时，调度器会对节点进行预选和优选两个阶段的评估。节点评估流程如图 5.5 所示。

在预选阶段，调度器会对每个节点 CPU、内存的使用率进行计算，若当前节点的 CPU、内存使用率已达到节点总资源的 80%，则将该节点过滤，从而避免在该节点上产生更多负载导致任务所需资源不足。

在优选阶段，首先，调度器会获取 Pod 中的容器创建信息，并根据所需镜像，从镜像仓库中获取其对应的 manifest 文件以构建所需的镜像层列表。随后从第三章所述的各节点 CR 对象中获取缓存的镜像层信息，最终通过公式(3-1)对该镜像的缓存命中率情况进行评分得到 *cacheHitScore*。

其次，根据 Pod 中各容器的资源需求（包括 CPU、内存）分别计算所需的资源总和，并依据节点可分配资源进行评分，评分的具体公式如下：

$$useRateCPU = (allocatedCPU + requestCPU) / allocatableCPU \quad (5-1)$$

$$useRateMem = (allocatedMem + requestMem) / allocatableMem \quad (5-2)$$

$$resourceScore = useRateCPU \times 50 + useRateMem \times 50 \quad (5-3)$$

其中 *useRateCPU* 和 *useRateMem* 分别代表了若将该 Pod 调度到此节点上，该节点的 CPU 和内存使用率，*allocatedCPU* 和 *allocatedMem* 代表了该节点已分配的 CPU 和内存资源，通过计算该节点上已存在 Pod 中容器的需求资源总和得到。*requestCPU* 和 *requestMem* 为待调度任务 CPU 和内存的需求量，该需求量在任务 yaml 文件的 *resources/requests* 字段指定。*allocatableCPU* 和 *allocatableMem* 代表节点可分配的总 CPU 和内存资源，该数值由节点硬件资源决定，通过与 API Server 交互查询。最终将二者进行加权得到节点的资源利用率评分 *resourceScore*。

此外，对于一些对于负载均衡要求较高的任务需求，使用默认调度器的资源平衡策略则更为适合，为了满足上述两种类型任务的场景需求，同时也使得镜像层缓存在上述两种场景中均得到有效利用，本模块将 Kubernetes 默认调度器的资源平衡策略得分 *balanceScore* 与上述两个评分结果进行加权融合，具体公式如下：

$$Score = cacheHitScore \times w_1 + resourceScore \times w_2 + balanceScore \times w_3 \quad (5-4)$$

其中 w_1 、 w_2 、 w_3 分别代表了任务对缓存命中率、单节点资源利用率以及负载均衡的需求程度。

在实际使用中，用户可针对具体场景对上述权重进行调整，从而满足不同类型任务需求的同时有效提升容器部署速度。例如在任务数量众多并对负载均衡需求不高的场景中，通过提高单节点资源利用率，可以使得任务更加集中地调度到较少的节点，减少了需要下载镜像的节点数量。而对于负载均衡要求较高的场景，即便此时工作节点数量会有所增加，但任务也会倾向于被调度到拥有所需镜像层的节点，相比于使用 Kubernetes 默认调度方案，本方案能够更充分地利用镜像层缓存。

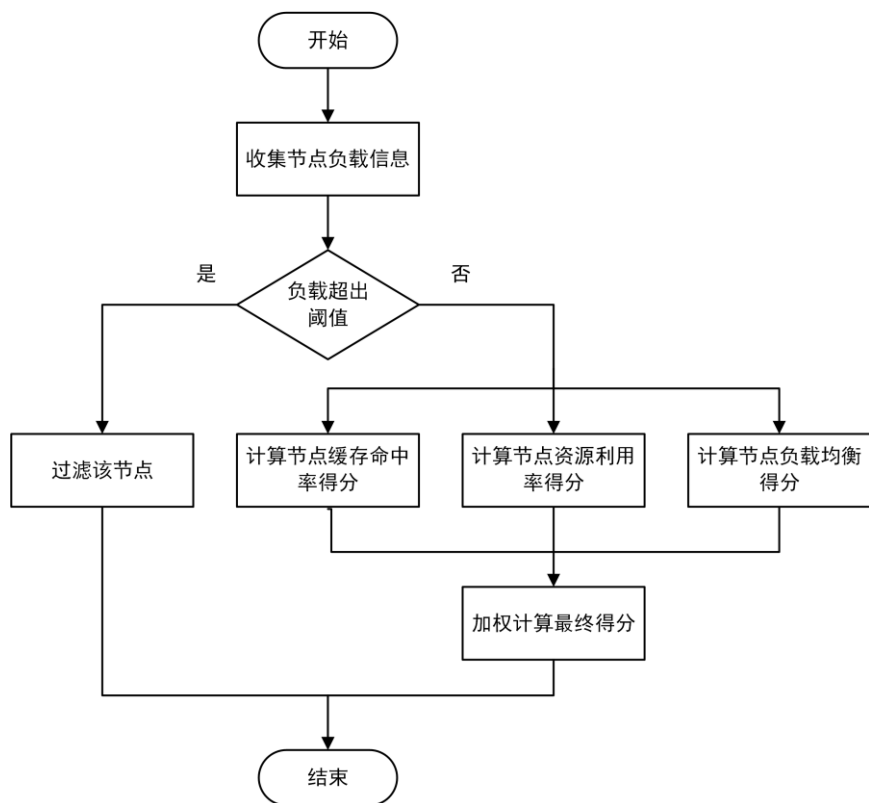


图 5.5 容器调度模块节点评分流程

5.2.3 镜像分发模块

镜像分发模块用于在容器调度完成后，根据集群情况自适应选择最优策略将容器所需镜像高效地分发到指定节点。在实际应用中，节点规模、所需镜像大小、镜像仓库的网络状况以及节点缓存量等因素均会在一定程度上影响镜像分发的总体时延，同时一些具有快速响应需求的任务也对部分节点的分发速度有着额外要求。本模块综合上述因素，在镜像传输方式、初始节点选择、镜像传输结构以及节点下载顺序上进行自动化方案决策，从而灵活适应多种任务场景、减少开发人员的操作量，同时准确高效地提升镜像分发效率。

该模块工作流程如图 5.6 所示。首先，该模块与 API Server 交互，依据任务标签从 Etcd 中读取 Pod 被调度到的节点的信息，并将上述节点添加到需求节点列表中。紧接着，该模块从镜像 manifest 文件中得知任务所需的镜像大小及镜像层信息，并与各节点的镜像层缓存进行对比。对于拥有缓存比例超过所设阈值的节点，使其从镜像仓库下载所需的剩余镜像层，同时记录下载的数据量以及时间，以此计算镜像仓库的网络负载状况。随后，该模块依据节点数量、所需镜像大小及镜像仓库网络负载选择镜像传输方式，当其中之一超过所设阈值时，由之前下载完整镜像的节点作为初始源节点，剩余节点通过 IPFS 进行镜像传输，否则剩余节点会继续从中心镜像仓库下载镜像。

此方案可以充分发挥镜像仓库和 IPFS 在不同场景中的优势，在大规模或大体积镜像分发任务中，通过 IPFS 传输避免了中心式下载的网络拥堵，并且挑选了缓存比例较高的节点优先下载并作为初始源节点，大大减少了初始源节点的准备时间，提升了整体的传输速率。而在任务规模和所需镜像较小且镜像仓库负载较低时，由于 IPFS 在文件共享、节点发现等过程中具有额外的时延开销，因此选择了直接从镜像仓库获取镜像以获得更高的效率。

若在上述流程中选择了 IPFS 镜像传输方式，该模块将继续在镜像的传输结构上进行决策。首先统计所有需求节点的镜像层缓存量，随后计算其相对于完整镜像的大小占比是否超过所设阈值，最终判断以镜像层为单位传输或是以完整镜像为单位传输。当节点缓存量较高时，通过以镜像层为单位传输，各节点只需获取各自所缺乏的层，从而减少了数据的重复下载。但与此同时，该过程需要将导出的镜像文件进行解压和分层，将每个镜像层分别进行共享，并且在传输后需要与本地已有的镜像层重构为完整镜像并导入。在面对缓存较少的情况时，分层传输所减少的下载量十分有限，并且由于需要额外的解压和重构时间，因此选择了传输完整镜像从而获得更高的效率。

此外在节点分发顺序上，该模块依据节点规模及任务需求决定是否在传输时集中带宽给部分节点以进行分批传输。若节点规模超出所设阈值，且任务只关注总体传输时延，则会以所有节点同时加入 IPFS 网络的方式进行传输；而若节点规模未超出阈值，或者任务更关注响应速度、平均启动时延或是传输时的稳定性，则会采用第四章图 4.3 所示方案将节点分批加入 IPFS 网络进行传输。

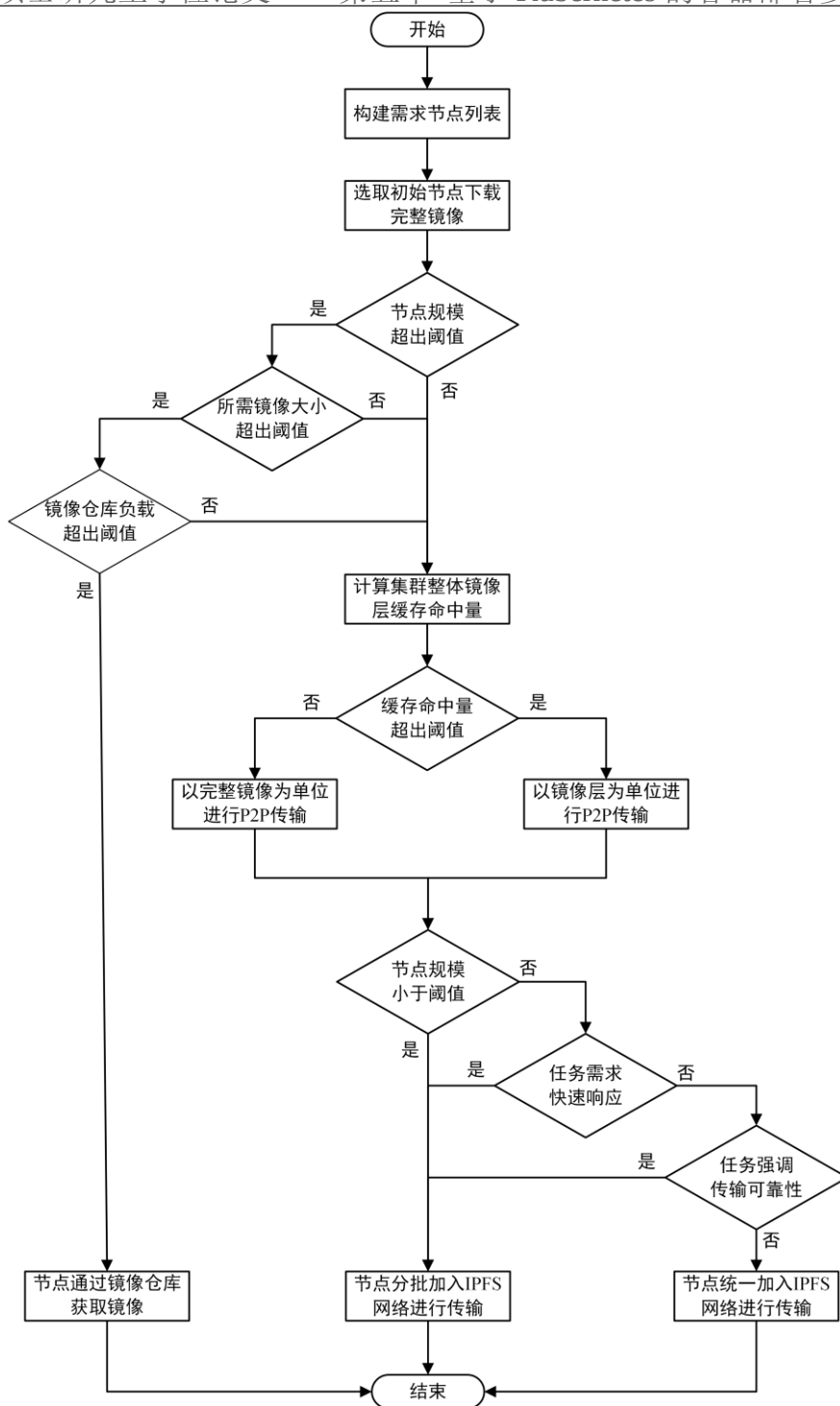


图 5.6 镜像分发模块工作流程

5.2.4 反馈与调节模块

反馈与调节模块用于在容器部署过程中，记录集群与镜像仓库的性能状态、调度结果以及部署时延，并通过日志的形式展示给用户，以便于监控和评估部署效率。此外，系统将各项权重与阈值通过 ConfigMap 暴露给用户，用户能够基于历史数据对系统参数进行细致调整，进而提升系统评估与决策的准确性。在面对不同任务需求时，用户也可以通过 ConfigMap 快

南京邮电大学硕士研究生学位论文 第五章 基于 Kubernetes 的容器部署多阶段加速系统

速调整部署方案，而无需反复修改系统镜像，从而保障系统的灵活性和操作的便捷性。该模块统计与反馈的具体指标如表 5.1 所示，用户可调节的权重及阈值如表 5.2 所示。

表 5.1 系统反馈指标

统计指标	说明
execution_time	任务创建时间
prepull_list	镜像预拉取列表
node_list	任务被调度的工作节点列表
image_size	任务所需镜像大小
registry_data_rate	镜像仓库数据传输速率
cache_rate	集群整体缓存命中率
deploy_latency	该次容器部署流程的总体时延

表 5.2 系统可调节的权重与阈值

权重与阈值	说明
cache_weight	节点缓存命中率评分权重
resource_weight	节点资源利用率评分权重
balance_weight	节点负载均衡评分权重
node_number_p2p_threshold	当节点数量高于此阈值时，通过 IPFS 传输镜像
image_size_threshold	当镜像大小高于此阈值时，通过 IPFS 传输镜像
registry_rate_threshold	当镜像仓库传输速率低于此阈值时，通过 IPFS 传输镜像
cache_rate_threshold	当集群整体缓存命中率高于此阈值时，以镜像层为单位传输
node_number_transfer_threshold	当节点数量低于此阈值时，采用节点分批传输策略
quick_response	任务是否需要快速响应
stable_transmission	任务是否需要优先保障传输稳定性

5.3 实验与分析

本节通过多组实验评估了上述容器部署系统在实际应用场景中的可行性和高效性。首先介绍了本系统的部署环境及相关硬件配置，随后对每个模块应用前后的部署效率进行测试，并与 Kubernetes 默认部署方式进行对比以验证系统性能的提升。

5.3.1 实验环境

本实验通过十台虚拟机搭建了集群环境，其中 master 为主控节点，用于管理集群与部署本文系统；worker 为工作节点，用于运行具体的任务容器；registry 为中心镜像仓库，存储了实验中所需的镜像。各节点的相关配置如表 5.3 所示。

表 5.3 实验环境

节点名称	系统	CPU 核数	内存 (GB)	带宽 (Mbps)
master	Centos7	4	6	1000
worker1~8	Centos7	2	4	1000
registry	Centos7	2	4	1000

上述节点统一配置了 Kubernetes 容器编排引擎，其版本为 1.19，各节点的容器运行时为 Docker 19.03。此外集群中已安装 IPFS 和 Prometheus 监控组件，并通过 DaemonSet 于各节点上部署了 Node Exporter 以收集节点信息。系统最终以容器化的方式部署于集群中，并已为其提供了与 API Server 交互所需的各项权限。

5.3.2 实验结果对比

首先，为验证镜像预拉取模块所带来的性能提升，本实验对比了集群在不同镜像层缓存比例下的容器部署时延。实验首先使用 Dockerfile 构建了一个大小为 100MB 基础镜像，随后在此基础镜像上依次添加了 1 到 4 个 100MB 的镜像层，形成多个测试镜像以模拟不同的缓存命中情况，其大小范围从 200MB 到 500MB 不等。在实验中，集群各节点通过镜像预拉取模块分别获取了上述 100MB 到 400MB 的对应镜像。并以最终以 500MB 的镜像为需求镜像，通过 DaemonSet 将容器调度在集群中的所有 worker 节点，在调度后，各节点从镜像仓库拉取缺失的镜像层以完成容器部署。实验结果如图 5.7 所示。

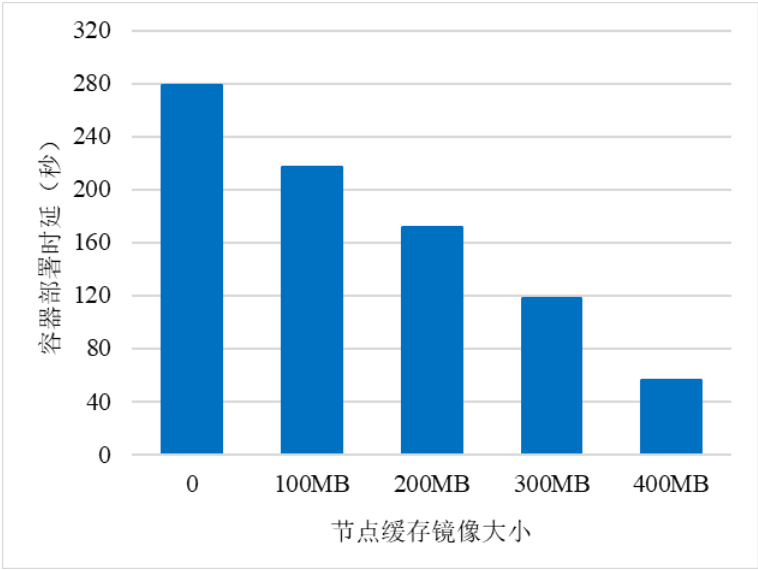


图 5.7 不同缓存命中下容器部署时延

结果表明，镜像的预拉取能够大幅减少任务到来时的容器部署时延，主要得益于在获取镜像时利用了本地已有的缓存，从而减少了下载量，其提升效果随镜像层命中率的增加而增

加。需要注意的是，本实验模拟的场景建立在预先知道所需镜像的前提下，在实际使用中，用户需要根据业务场景，准确判断未来可能使用到的镜像缓存，从而达到最理想效果。

其次，在镜像预拉取的基础上，本实验进一步验证容器调度模块所带来的性能提升。在对资源利用率和负载均衡具有需求的任务场景中，分别对比了使用本系统的调度方式和使用 Kubernetes 默认调度方式的容器部署时延。在实验中，任务通过 Deployment 设置了多个不同的副本数量，同时每个 Pod 具有 0.5 核 CPU 和 1GB 内存需求，所需镜像为 500MB 的自制镜像，统一从镜像仓库下载，并在 worker1 和 worker2 节点预先缓存了 60%所需镜像层，在 worker3 和 worker4 节点预先缓存了 40%所需镜像层。实验结果如图 5.8 所示。

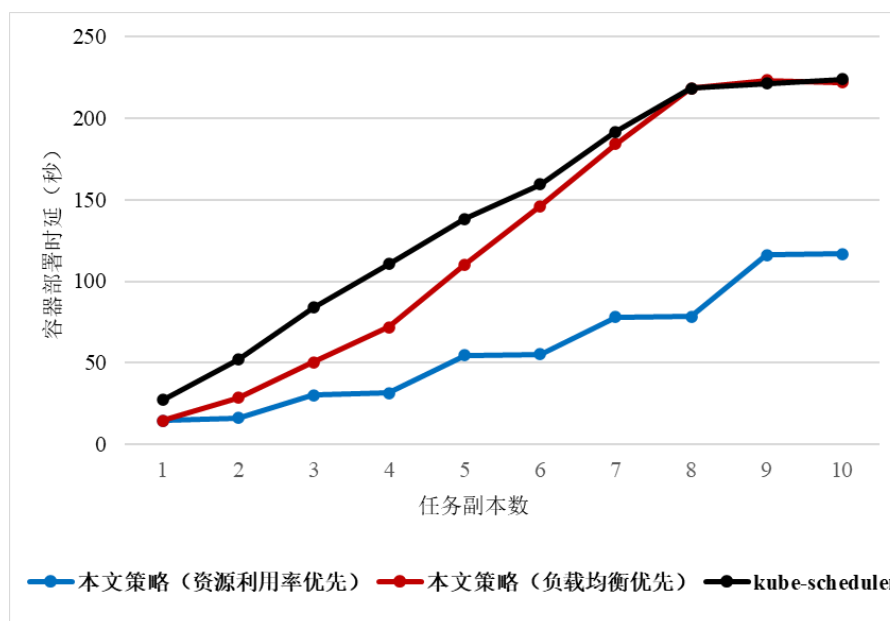


图 5.8 不同任务需求下容器部署时延对比

结果表明，在面向上述两种不同场景需求的任务时，使用本调度模块均能有效提升容器部署效率。在面向具有资源利用率需求的任务时，容器优先被集中调度到具有一定缓存量的节点，仅在超出节点负载上限时，才会被调度到更多其他节点。相比于默认调度方式，本模块有效减少了下载镜像的节点数量，从而降低了部署时延。在面向具有负载均衡需求的任务时，若多个节点具有相近的负载状况，本模块会优先选择节点缓存较高的节点从而减少了下载量，提升了部署速度。仅在任务副本数较多时，为平衡负载，容器被分散到了集群中的所有节点，从而和默认调度方式具有相近的容器部署速度。

最后，本实验在上述模块的基础上，继续测试了镜像分发模块所带来的性能提升。由于在本文第四章实验中，已经详细对比并分析了在不同节点规模下，使用不同镜像分发方式的总体和平均传输速度。因此本实验主要聚焦于该模块在最优初始节点选择和以镜像层为单位进行 P2P 传输上的性能提升。本实验在 worker1~worker6 节点分发上述 500MB 的测试镜像，同时 worker1 和 worker2 节点上拥有 60%所需镜像缓存，其余节点拥有 20%所需镜像缓存。

南京邮电大学硕士研究生学位论文 第五章 基于 Kubernetes 的容器部署多阶段加速系统

本文设置了多组对照实验，各组镜像分发方式如表 5.4 所示。其中，采用 P2P 分发镜像的对照组均使用本文第四章图 4.3 方案对节点进行分批传输。实验结果如图 5.9 所示。

表 5.4 实验对照组设置

对照组	镜像分发方式	初始节点选择	传输单位
实验 A	镜像仓库下载	\	\
实验 B	P2P 传输	随机选取	完整镜像
实验 C	P2P 传输	依据缓存选取	完整镜像
实验 D	P2P 传输	随机选取	镜像层
实验 E	P2P 传输	依据缓存选取	镜像层

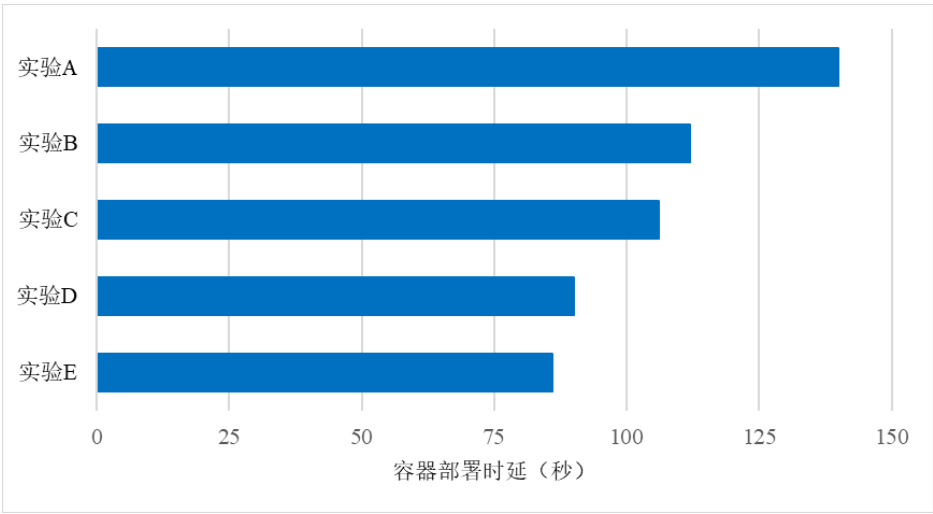


图 5.9 不同分发方案下容器部署时延对比

对比实验 A 和实验 B 可知，在多节点并发拉取时，采用 P2P 传输能够有效减少中心式拉取的带宽竞争，进而提升分发速度。然而在实验 B 中，由于传输的是完整镜像，导致了节点缓存的浪费，而实验 A 从镜像仓库获取镜像时，仅需下载所缺乏的镜像层，因此实验 B 相较于实验 A 的性能提升并未达到最理想幅度。对比实验 B 和实验 C 可知，当选取缓存量较多的节点作为初始节点时，能够有效地提升初始节点的准备速度，进而减少总体分发时延。对比实验 B 和实验 D 可知，在节点拥有较多缓存时，采用以镜像层为单位的 P2P 传输能够有效减少传输量，提升分发速度。在上述实验环境中，本模块最终自适应地采用了实验 E 方案，其综合了实验 C 和实验 D 方案的优势，在分发速度上得到了进一步提升。

综上所述，本文系统对容器部署过程中的多个阶段进行了优化，在任务到来前预先构建节点缓存并在调度时充分利用上述缓存，减少了镜像下载量；在镜像分发时依据任务需求和集群环境，自适应选择高效的分发方式，加速了镜像下载。实验结果表明，本系统能够在满足不同任务需求的前提下，有效提升容器部署速度。

5.4 本章小结

本章在本文第三、四章相关工作的基础上，设计了一套云原生环境下基于 Kubernetes 的容器部署多阶段加速系统。首先分析了在实际应用场景中，加速容器部署、降低操作复杂性的重要意义，指出了本系统的功能和非功能需求。随后展示了本文系统的总体概览，详细阐述了各模块的设计与实现，最后通过一系列实验对比，验证了本系统在容器部署速度上的性能提升。

第六章 总结与展望

6.1 论文研究工作总结

随着云计算技术的发展,云原生架构逐渐成为现代软件开发与部署的核心。在此环境下,为快速扩展服务并提升运行效率,业界普遍将应用程序以容器化的方式运行。然而在实际应用中,容器部署面临着一系列挑战。在容器调度时,现有方案往往难以在满足不同任务需求的同时兼顾效率指标。在镜像分发时,传统的分发机制在面对多并发拉取请求和复杂网络环境时,经常遇到效率瓶颈。此外,在整体的部署过程中,目前缺乏一套基于多方面因素进行自适应调整的流程化系统,因此难以保障容器部署的灵活性和鲁棒性。

针对上述问题,本文对云原生环境下容器部署加速策略进行了研究,提出了在容器部署过程中的多个阶段的优化方案。最终在 Kubernetes 容器编排引擎的基础上,设计并实现了一套完整的容器部署多阶段加速系统。本文的具体工作如下:

(1) 设计了一种基于节点镜像层缓存的容器调度方案。首先将节点镜像层缓存信息构建为集群的自定义资源,并通过自定义控制器对节点缓存信息进行获取及更新。随后基于上述工作设计了一个自定义调度器,当任务来临时,调度器会依据任务所需的镜像层,判断各节点的镜像层缓存命中率,并将任务调度到缓存命中率最高的节点上。从而在后续的镜像分发阶段减少剩余镜像层的下载量,提高容器部署速度。相比于 Kubernetes 的本地存储优先策略,本方案有着更小的细粒度,能够更高效地利用节点缓存。

(2) 设计了一种基于 IPFS 的分批式镜像分发方案。该方案通过 IPFS 进行镜像 P2P 传输,有效缓解了镜像仓库在高并发下载请求时的网络拥堵,同时相比于 BitTorrent 传输方式,其无需依赖 Tracker 服务器并且避免了种子文件的额外开销。在此基础上,针对其节点发现耗时久、任务响应速度慢和鲁棒性欠缺的问题,设计了一种节点分批传输策略,该策略优先保障了部分节点获取完整镜像并在传输的过程中逐步提升网络规模,从而降低了网络结构的复杂度,减少了节点发现时延。同时部分节点能够更快启动,提高了任务平均响应速度。此外当初始节点出现故障时,剩余的节点也可以从其他拥有完整副本的节点中获取镜像,从而提升了传输的鲁棒性。

(3) 设计了一种云原生环境下基于 Kubernetes 的容器部署多阶段加速系统。系统在集群网络负载较低时预先下载可能使用到的镜像,有效提升了任务到来时的节点缓存利用率。在容器调度时,系统在满足不同任务需求的前提下,高效地利用了节点的缓存,减少了后续镜

像的下载量。在镜像分发时，系统可以自适应地调整镜像分发方式，从而在面对不同集群环境和任务类型时，保持较高的镜像分发速度。在完整部署流程结束后，系统可以准确地向用户反馈部署过程中的各项指标，并提供了统一接口使用户能够对系统参数进行调优，从而降低了操作复杂度，提高了系统运作效率。

6.2 进一步研究工作展望

本文研究和分析了容器部署过程中的一系列优化方案，并将其融合构建为完整的系统。本文工作虽然已经取得了一定成效，但仍然有许多改进的空间：

（1）由于硬件条件的限制，本文实验最多模拟只能八个工作节点，因此在设计镜像分批传输策略时未考虑节点规模因素。在未来测试条件允许时，计划对该策略进行调整，在传输过程中依据源节点数以及总需求节点数，合理调节各批次传输的节点数量，从而减少后续节点的等待时间，提升其在大规模集群场景下的性能。

（2）在调度阶段，本文系统目前只考虑了缓存命中、资源利用率和负载均衡的相关指标，未来计划加入存储、网络带宽等性能参数，从而适应更多类型的任务需求，进一步提升系统的灵活性。

（3）在部署完成后，虽然本系统提供了详细的历史数据以便开发人员对系统参数进行调整，但其过多依赖于人为的主观经验。因此未来计划借助机器学习技术^[75]，依据历史系统参数和结果进行模型训练，最终实现系统的自动调优，从而提高优化过程的准确性，减少人工的操作量。

参考文献

- [1] Matthew U O, Kazaure J S, Okafor N U. Contemporary development in E-Learning education, cloud computing technology & internet of things[J]. EAI Endorsed Transactions on Cloud Systems, 2021, 7(20): e3-e3.
- [2] Dittakavi R S S K. Achieving the Delicate Balance: Resource Optimization and Cost Efficiency in Kubernetes[J]. Eduzone: International Peer Reviewed/Refereed Multidisciplinary Journal, 2023, 12(2): 125-131.
- [3] Brady K, Moon S, Nguyen T, et al. Docker container security in cloud computing[C]. 10th Annual Computing and Communication Workshop and Conference (CCWC). IEEE, 2020: 0975-0980.
- [4] Bazm M M, Lacoste M, Südholt M, et al. Isolation in cloud computing infrastructures: new security challenges[J]. Annals of Telecommunications, 2019, 74: 197-209.
- [5] Bhardwaj A, Krishna C R. Virtualization in cloud computing: Moving from hypervisor to containerization—a survey[J]. Arabian Journal for Science and Engineering, 2021, 46(9): 8585-8601.
- [6] Rashid A, Chaturvedi A. Cloud computing characteristics and services: a brief review[J]. International Journal of Computer Sciences and Engineering, 2019, 7(2): 421-426.
- [7] Ramchand K, Baruwat Chhetri M, Kowalczyk R. Enterprise adoption of cloud computing with application portfolio profiling and application portfolio assessment[J]. Journal of Cloud Computing, 2021, 10(1): 1-18.
- [8] Katal A, Dahiya S, Choudhury T. Energy efficiency in cloud computing data centers: a survey on software technologies[J]. Cluster Computing, 2023, 26(3): 1845-1875.
- [9] Karthikeyan K, Sunder R, Shankar K, et al. Energy consumption analysis of Virtual Machine migration in cloud using hybrid swarm optimization (ABC-BA)[J]. The Journal of Supercomputing, 2020, 76: 3374-3390.
- [10] Yang C T, Liu J C, Chen S T, et al. Virtual machine management system based on the power saving algorithm in cloud[J]. Journal of Network and Computer Applications, 2017, 80: 165-180.
- [11] He T Z, Toosi A N, Buyya R. Camig: Concurrency-aware live migration management of multiple virtual machines in sdn-enabled clouds[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 33(10): 2318-2331.
- [12] Sandhu A K. Big data with cloud computing: Discussions and challenges[J]. Big Data Mining and Analytics, 2021, 5(1): 32-40.
- [13] Belgaum M R, Alansari Z, Musa S, et al. Role of artificial intelligence in cloud computing, IoT and SDN: Reliability and scalability issues[J]. International Journal of Electrical and Computer Engineering, 2021, 11(5): 4458-4470.
- [14] Bhardwaj A, Rama Krishna C. A container-based technique to improve virtual machine migration in cloud computing[J]. IETE Journal of Research, 2022, 68(1): 401-416.
- [15] Bentaleb O, Belloum A S Z, Sebaa A, et al. Containerization technologies: Taxonomies, applications and challenges[J]. The Journal of Supercomputing, 2022, 78(1): 1144-1181.
- [16] Potdar A M, Narayan D G, Kengond S, et al. Performance evaluation of docker container and virtual machine[J]. Procedia Computer Science, 2020, 171: 1419-1428.
- [17] Maenhaut P J, Volckaert B, Ongenae V, et al. Resource management in a containerized cloud: Status and challenges[J]. Journal of Network and Systems Management, 2020, 28: 197-246.
- [18] Shah J, Dubaria D. Building modern clouds: using Docker, Kubernetes & Google cloud platform[C]. IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC). IEEE, 2019: 0184-0189.
- [19] Song S, Deng L, Gong J, et al. Gaia scheduler: A Kubernetes-based scheduler framework[C]. IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications

- (ISPA/IUCC/BDCloud/SocialCom/SustainCom). IEEE, 2018: 252-259.
- [20] Nguyen Q M, Phan L A, Kim T. Load-balancing of Kubernetes-based edge computing infrastructure using resource adaptive proxy[J]. *Sensors*, 2022, 22(8): 2869-2878.
- [21] Fu Y, Zhang S, Terrero J, et al. Progress-based container scheduling for short-lived applications in a Kubernetes cluster[C]. *IEEE International Conference on Big Data (Big Data)*. IEEE, 2019: 278-287.
- [22] Rejiba Z, Chamanara J. Custom scheduling in Kubernetes: a survey on common problems and solution approaches[J]. *ACM Computing Surveys*, 2022, 55(7): 1-37.
- [23] Anwar A, Mohamed M, Tarasov V, et al. Improving Docker registry design based on production workload analysis[C]. *16th USENIX Conference on File and Storage Technologies (FAST 18)*. 2018: 265-278.
- [24] Harter T, Salmon B, Liu R, et al. Slacker: Fast distribution with lazy Docker containers[C]. *14th USENIX Conference on File and Storage Technologies (FAST 16)*. 2016: 181-195.
- [25] Zhang D, Yan B H, Feng Z, et al. Container oriented job scheduling using linear programming model[C]. *3rd International Conference on Information Management (ICIM)*. IEEE, 2017: 174-180.
- [26] Liu B, Li P, Lin W, et al. A new container scheduling algorithm based on multi-objective optimization[J]. *Soft Computing*, 2018, 22: 7741-7752.
- [27] Guo Y, Yao W. A container scheduling strategy based on neighborhood division in micro service[C]. *IEEE/IFIP Network Operations and Management Symposium (NOMS)*. IEEE, 2018: 1-6.
- [28] Menouer T, Darmon P. New scheduling strategy based on multi-criteria decision algorithm[C]. *27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. IEEE, 2019: 101-107.
- [29] Guo W, Huang H, Niu Z, et al. A Task Priority-based Resource Scheduling Algorithm for Container-based Clouds[C]. *IEEE International Conference on Emergency Science and Information Technology (ICESIT)*. IEEE, 2021: 268-273.
- [30] 常旭征, 焦文彬. Kubernetes 资源调度算法的改进与实现[J]. *计算机系统应用*, 2020, 29(7): 256-259.
- [31] Santos J, Wauters T, Volckaert B, et al. Towards network-aware resource provisioning in kubernetes for fog computing applications[C]. *IEEE Conference on Network Softwarization (NetSoft)*. IEEE, 2019: 351-359.
- [32] 唐瑞. 基于 Kubernetes 的容器云平台资源调度策略研究[D]. 电子科技大学, 2017.
- [33] Fu S, Mittal R, Zhang L, et al. Fast and efficient container startup at the edge via dependency scheduling[C]. *3rd USENIX Workshop on Hot Topics in Edge Computing (HotEdge 20)*. 2020: 1-7.
- [34] Kangjin W, Yong Y, Ying L, et al. FID: A faster image distribution system for Docker platform[C]. *IEEE 2nd International Workshops on Foundations and Applications of Self* Systems (FAS* W)*. IEEE, 2017: 191-198.
- [35] Liang M, Shen S, Li D, et al. HDID: An efficient hybrid docker image distribution system for datacenters[C]. *15th National Software Application Conference (NASAC)*. 2016: 179-194.
- [36] 张浩, 孙毓忠, 肖立等. RainbowD: 一种异构云环境下高效的 Docker 镜像分发系统[J]. *计算机学报*, 2020, 43(11): 2067-2083.
- [37] 贺龙. 大规模容器云平台中的镜像分发与资源调度系统的研究与实现[D]. 北京邮电大学, 2021.
- [38] 姚增增. 大规模环境下容器技术的研究与优化[D]. 浙江大学, 2019.
- [39] Du L, Wo T, Yang R, et al. Cider: A rapid docker container deployment system through sharing network storage[C]. *IEEE 19th International Conference on High Performance Computing and Communications; IEEE 15th International Conference on Smart City; IEEE 3rd International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*. IEEE, 2017: 332-339.
- [40] Nathan S, Ghosh R, Mukherjee T, et al. Comicon: A co-operative management system for docker container images[C]. *IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2017: 116-126.
- [41] Zheng C, Rupprecht L, Tarasov V, et al. Wharf: Sharing docker images in a distributed file system[C]. *Proceedings of the ACM Symposium on Cloud Computing*. 2018: 174-185.
- [42] Lee H, Fox G C. Efficient Software Defined Systems using Common Core Components[C]. *IEEE 10th International Conference on Cloud Computing (CLOUD)*. IEEE, 2017: 407-414.

- [43] 周毅, 邓玉辉. 一种概率模型的 Docker 镜像删减策略[J]. 小型微型计算机系统, 2018, 39(09): 1908-1913.
- [44] Li H, Yuan Y, Du R, et al. {DADI}:{Block-Level} Image Service for Agile and Elastic Application Deployment[C]. USENIX Annual Technical Conference (USENIX ATC 20). 2020: 727-740.
- [45] Huang Z, Wu S, Jiang S, et al. Fastbuild: Accelerating docker image building for efficient development and deployment of container[C]. 35th Symposium on Mass Storage Systems and Technologies (MSST). IEEE, 2019: 28-37.
- [46] Gotanda S, Shinagawa T. Short Paper: Highly Compatible Fast Container Startup with Lazy Layer Pull[C]. IEEE International Conference on Cloud Engineering (IC2E). IEEE, 2021: 53-59.
- [47] Mosciatti S, Lange C, Blomer J. Increasing the execution speed of containerized analysis workflows using an image snapshotter in combination with CVMFS[J]. Frontiers in big Data, 2021, 4: 673163.
- [48] Thalheim J, Bhatotia P, Fonseca P, et al. Cntr: Lightweight OS containers[C]. USENIX Annual Technical Conference (USENIX ATC 18). 2018: 199-212.
- [49] 卞盛伟. 基于索引结构的容器镜像系统的设计与实现[D]. 华中科技大学, 2022.
- [50] Ahmed A, Pierre G. Docker container deployment in fog computing infrastructures[C]. IEEE International Conference on Edge Computing (EDGE). IEEE, 2018: 1-8.
- [51] Kim J B, Kim J I. A Study of Application Development Method for Improving Productivity on Cloud Native Environment[J]. Journal of Korea Multimedia Society, 2020, 23(2): 328-342.
- [52] Li S, Zhang H, Jia Z, et al. Understanding and addressing quality attributes of microservices architecture: A Systematic literature review[J]. Information and software technology, 2021, 131: 106449.
- [53] De Lauretis L. From monolithic architecture to microservices architecture[C]. IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW). IEEE, 2019: 93-96.
- [54] Casalicchio E. Container orchestration: A survey[J]. Systems Modeling: Methodologies and Tools, 2019: 221-235.
- [55] Singh A, Singh V, Aggarwal A, et al. Improving Business deliveries using Continuous Integration and Continuous Delivery using Jenkins and an Advanced Version control system for Microservices-based system[C]. 5th International Conference on Multimedia, Signal Processing and Communication Technologies (IMPACT). IEEE, 2022: 1-4.
- [56] Dragonfly. 阿里巴巴镜像分发平台[EB/OL]. <https://d7y.io/zh/docs/next/>.
- [57] Docker[EB/OL]. <https://docs.docker.com/>.
- [58] Open Container Initiative[EB/OL]. <https://opencontainers.org/about/overview/>.
- [59] Nyabuto M G M, Mony M V, Mbugua S. Architectural Review of Client-Server Models[J]. International Journal of Scientific Research and Engineering Trends, 2023, 10(1): 139-143.
- [60] Hindman, Benjamin, et al. Mesos: A platform for fine-grained resource sharing in the data center[C]. 8th USENIX Symposium on Networked Systems Design and Implementation. Boston, USA: NSDI, 2011, 11: 22-36.
- [61] Naik N. Performance evaluation of distributed systems in multiple clouds using docker swarm[C]. 2021 IEEE International Systems Conference (SysCon). IEEE, 2021: 1-6.
- [62] Singh N, Hamid Y, Juneja S, et al. Load balancing and service discovery using Docker Swarm for microservice based big data applications[J]. Journal of Cloud Computing, 2023, 12(1): 1-9.
- [63] Kubernetes[EB/OL]. <https://kubernetes.io/docs/concepts/>.
- [64] Androutsellis-Theotokis S, Spinellis D. A survey of peer-to-peer content distribution technologies[J]. ACM computing surveys (CSUR), 2004, 36(4): 335-371.
- [65] Xia R L, Muppala J K. A survey of bittorrent performance[J]. IEEE Communications surveys & tutorials, 2010, 12(2): 140-158.
- [66] Muralidharan S, Ko H. An InterPlanetary file system (IPFS) based IoT framework[C]. IEEE international

- conference on consumer electronics (ICCE). IEEE, 2019: 1-2.
- [67] Ahmad I, AlFailakawi M G, AlMutawa A, et al. Container scheduling techniques: A survey and assessment[J]. Journal of King Saud University-Computer and Information Sciences, 2022, 34(7): 3934-3947.
- [68] Zhong Z, Buyya R. A cost-efficient container orchestration strategy in kubernetes-based cloud computing infrastructures with heterogeneous resources[J]. ACM Transactions on Internet Technology (TOIT), 2020, 20(2): 1-24.
- [69] Hadeed W, Abdullah D B. Load balancing mechanism for edge-cloud-based priorities containers[J]. Int. J. Wirel. Microwave Technol.(IJWMT), 2022, 12(5): 1-9.
- [70] Lin C, Nadi S, Khazaei H. A large-scale data set and an empirical study of docker images hosted on docker hub[C]. IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 2020: 371-381.
- [71] Pawlik R, Werewka J. Recreation of containers for high availability architecture and container-based applications[C]. Computer Networks: 26th International Conference, CN 2019, Kamień Śląski, Poland, June 25–27, 2019, Proceedings 26. Springer International Publishing, 2019: 287-298.
- [72] Kaliappan V K, Yu S, Soundararajan R, et al. High-secured data communication for cloud enabled secure docker image sharing technique using blockchain-based homomorphic encryption[J]. Energies, 2022, 15(15): 5544.
- [73] Becker S, Schmidt F, Kao O. Edgepier: P2p-based container image distribution in edge computing environments[C]. IEEE International Performance, Computing, and Communications Conference (IPCCC). IEEE, 2021: 1-8.
- [74] Prometheus[EB/OL]. <https://prometheus.io/docs/introduction/overview/>.
- [75] Cai L, Qi Y, Wei W, et al. Improving resource usages of containers through auto-tuning container resource parameters[J]. IEEE Access, 2019, 7: 108530-108541.